Natural Language Processing and LLMs
Lecture 05: Machine Translation

Valentin Malykh, MWS AI



Autumn 2026

The course is delivered at MIPT, HSE (Moscow), ITMO (Saint Petersburg), and IITU (Almaty)

Content

1. Machine Translation (MT)
2. Machine translation evaluation
3. Statistical machine translation (SMT)
4. Neural machine translation (NMT) based on RNNs
5. Neural machine translation (NMT) with attention

Content

1 Machine Translation (MT)

2 Machine translation evaluation

3 Statistical machine translation (SMT)

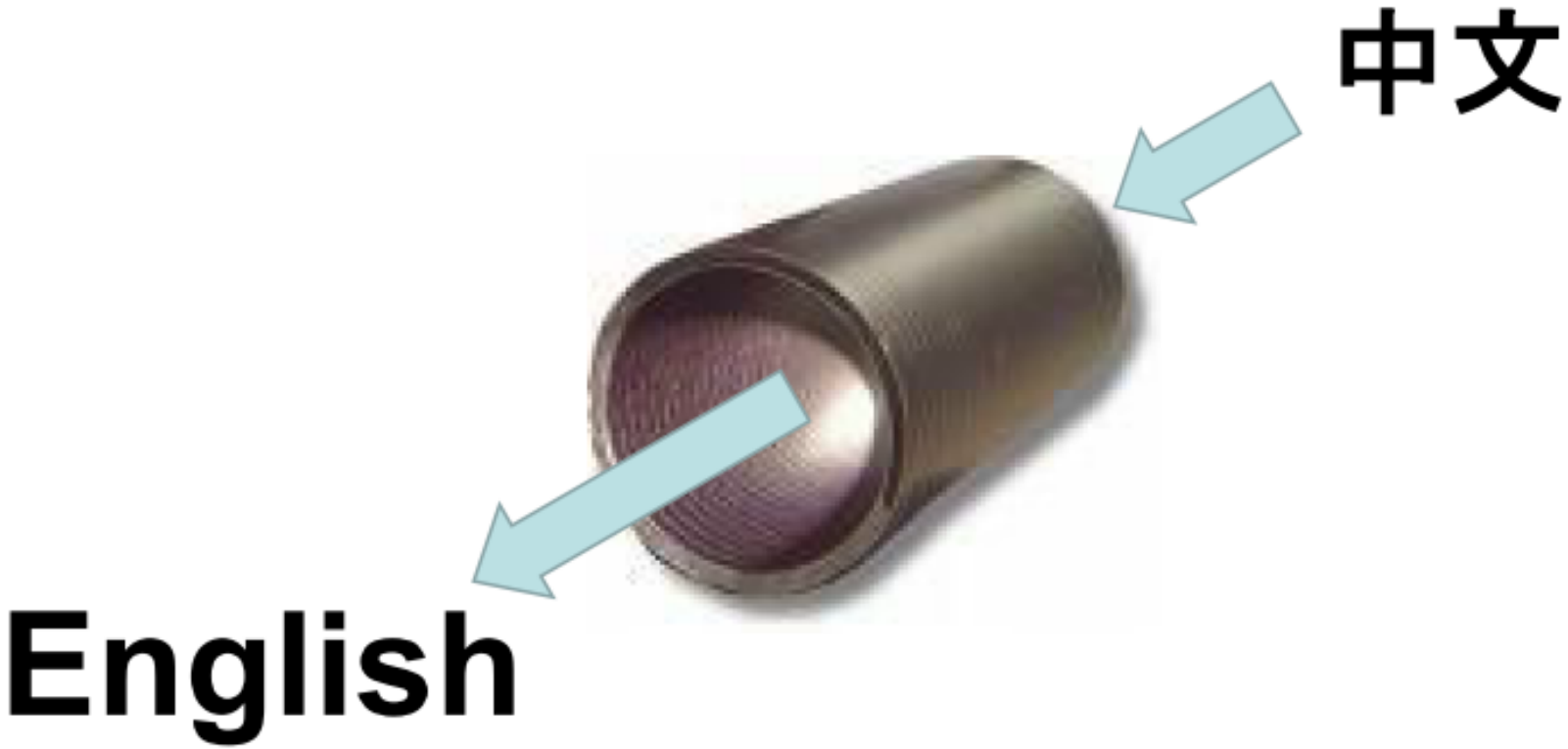
4 Neural machine translation (NMT) based on RNNs

5 Neural machine translation (NMT) with attention

Holy grails of NLP (Recap)

- **Accurate machine translation between human languages**
- Free conversation between humans and computers

Accurate machine translation (Recap)



A figure from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

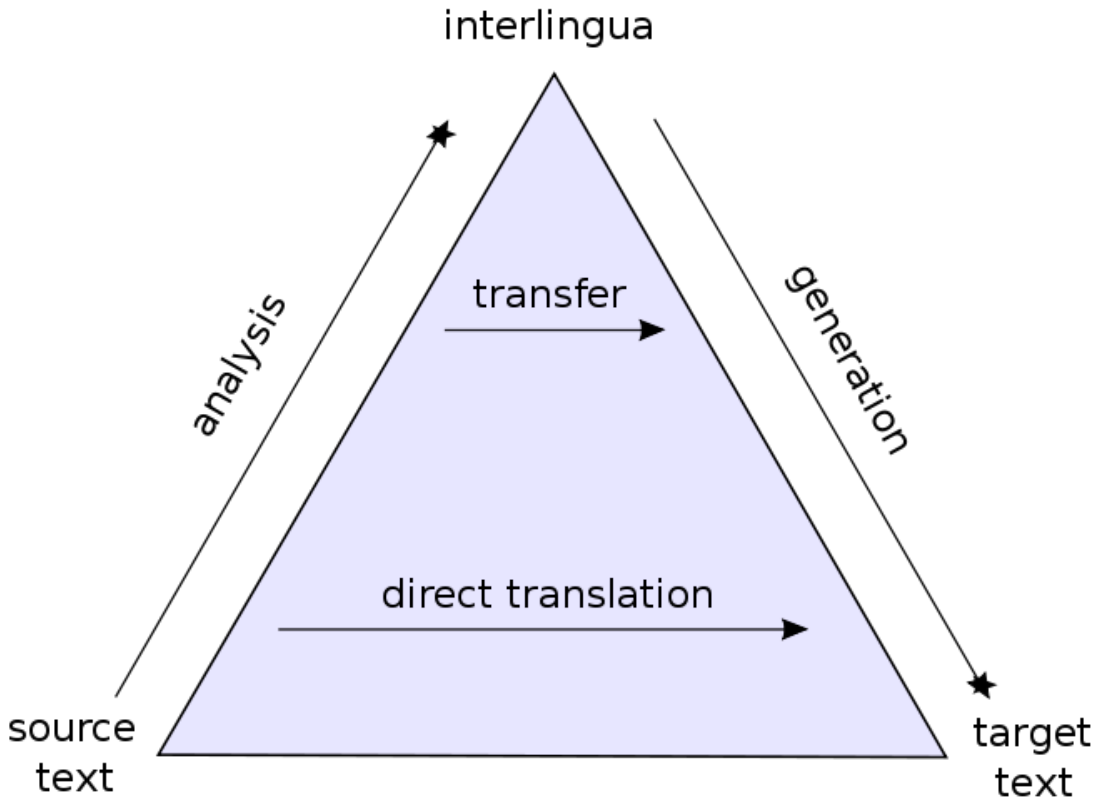
A brief history of MT

- 1940s–1950s: Early systems
- 1960s: ALPAC report (1966)
- 1970s: Operational systems: Systran, METEO
- 1980s: Eurotra project (EC-funded)
- Late 1980s–late 1990s: the dawn of SMT (IBM), EBMT
- 2000s–early 2010s: dominance of SMT
  - free on-line translation
- 2014–2017: Neural MT (RNN seq2seq with attention)
- 2017–present: Transformer-based NMT
- 2023–present: LLM-based translation

Different Approaches to MT

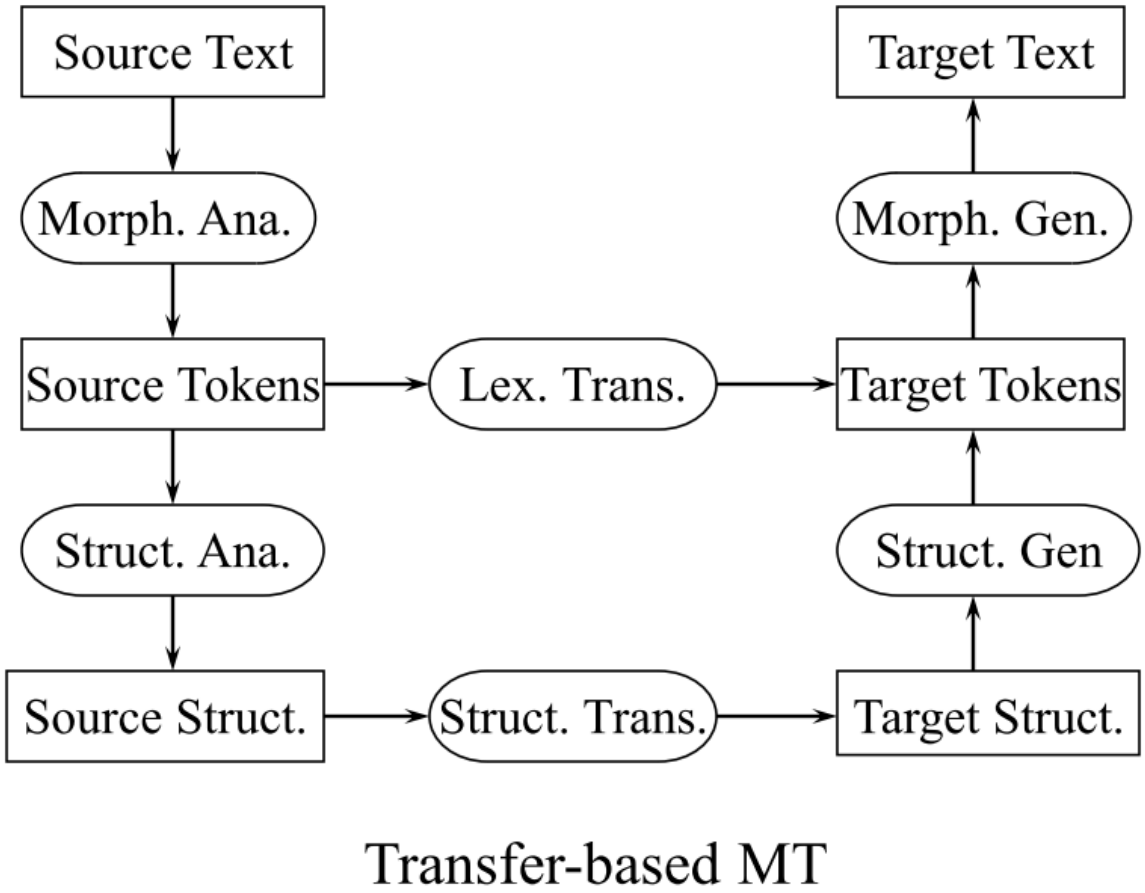
- Direct Translation
- Rule-based Machine Translation (RBMT)
- Memory-based Translation
- Example-based Machine Translation (EBMT)
- Statistical Machine Translation (SMT)
- Neural Machine Translation (NMT)

The Vauquois Pyramid



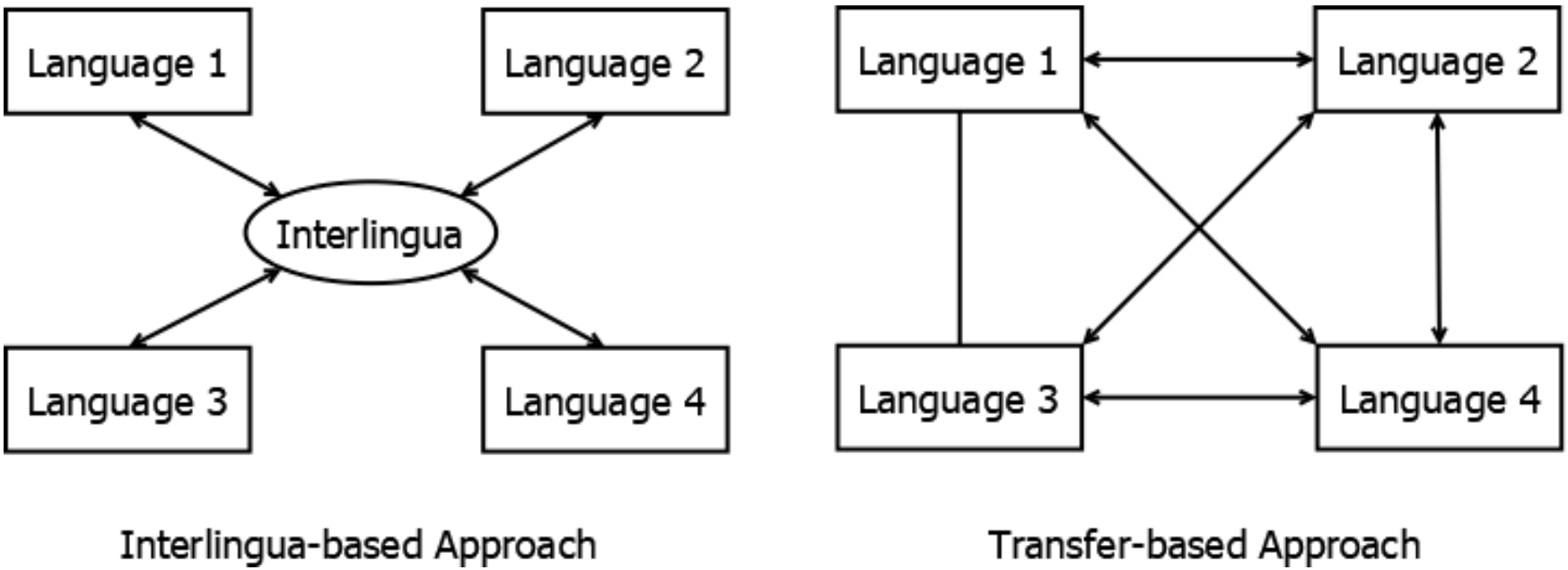
By Marajozkee, CC BY-SA 3.0, from Wikimedia Commons (recoloured)

Transfer-based MT



A figure from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

Interlingua approach vs. transfer approach



A figure from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

Multilingual MT

- Theoretically, interlingua-based approach is ideal because it requires far fewer components than the transfer-based approach.
- However, using a human-defined interlingua (i.e. a specific knowledge expression) is practically too difficult.
- In business, a feasible way is to use a natural language (mostly English) as the interlingua, which is usually called a pivot language in this case.

Multilingual MT

- Theoretically, interlingua-based approach is ideal because it requires far fewer components than the transfer-based approach.
- However, using a human-defined interlingua (i.e. a specific knowledge expression) is practically too difficult.
- In business, a feasible way is to use a natural language (mostly English) as the interlingua, which is usually called a pivot language in this case.

Multilingual MT

Makoto Nagao (Kyoto University):
When the pivot language (i.e. interlingua) is used, the results of the analytic stage must be in a form which can be utilized by all of the different languages into which translation is to take place. This level of subtlety is a practical impossibility. (Machine Translation, Oxford, 1989)

Multilingual MT

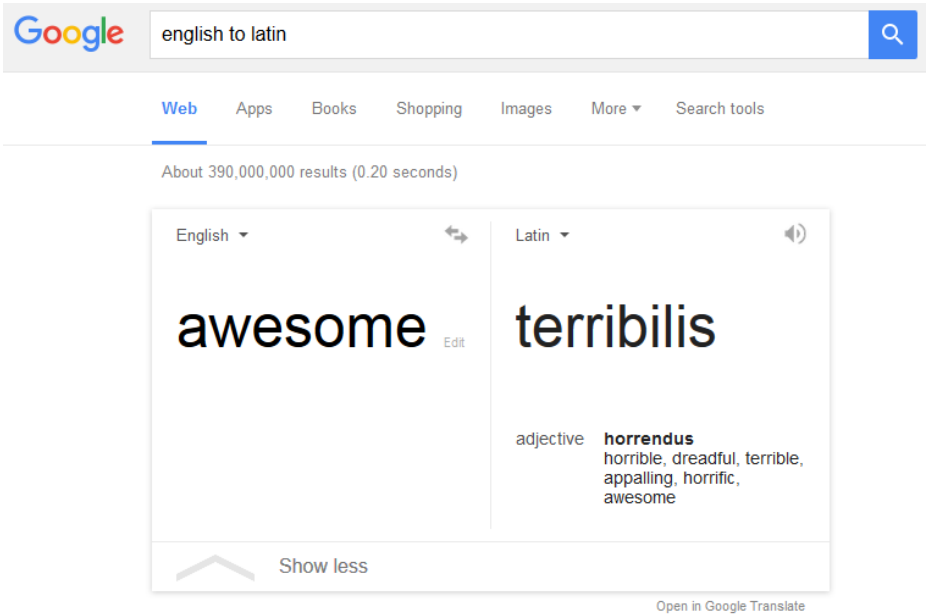
METAL system (Siemens):
METAL employs a modified transfer approach rather than an interlingua. If a meta-language (an interlingua) were to be used for translation purposes, it would need to incorporate all possible features of many languages. That would not only be an endless task but probably a fruitless one as well. Such a system would soon become unmanageable and perhaps collapse under its own weight.

Multilingual MT

- Theoretically, interlingua-based approach is ideal because it requires far fewer components than the transfer-based approach.
- However, using a human-defined interlingua (i.e. a specific knowledge expression) is practically too difficult.
- In business, a feasible way is to use a natural language (mostly English) as the interlingua, which is usually called a pivot language in this case.

Applications of MT

Since neural machine translation became popular, the quality of MT systems has greatly improved, and MT is used in more and more scenarios.



Web Translator

Figures from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

Applications of MT

Since neural machine translation became popular, the quality of MT systems has greatly improved, and MT is used in more and more scenarios.



Photo Translator

Figures from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

Applications of MT

Since neural machine translation became popular, the quality of MT systems has greatly improved, and MT is used in more and more scenarios.



Voice Translator

Figures from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

Applications of MT

Since neural machine translation became popular, the quality of MT systems has greatly improved, and MT is used in more and more scenarios.



Real-time Conference Translator

Figures from Qun Liu, Natural Language Processing, Huawei Noah's Ark Lab

Content

1 Machine Translation (MT)

2 Machine translation evaluation

3 Statistical machine translation (SMT)

4 Neural machine translation (NMT) based on RNNs

5 Neural machine translation (NMT) with attention

Machine translation evaluation is not easy

- Machine translation evaluation is not easy because correct translations for the same text may differ greatly in their wording.
- Human evaluation:
  - Reliable
  - Inconsistent
  - Expensive
  - Slow
- Automatic evaluation:
  - Unreliable
  - Consistent
  - Cheap
  - Fast

Goals for Automatic MT Evaluation

- Meaningful: score should give intuitive interpretation of translation quality.
- Consistent: repeated use of metric should give same results.
- Correct: metric must rank better systems higher.
- Low cost: reduce time and money spent on carrying out evaluation.
- Tunable: automatically optimise system performance towards metric.

Automatic MT Evaluation Metrics

- Input:
  - output of machine translation systems.
  - reference translations given by human translators.
  - source text (optional).
- Output:
  - a score which represents the similarity between the MT output and the human reference given the source sentence.

Automatic MT Evaluation Metrics

Plenty of automatic MT evaluation metrics have been proposed by researchers, among which the most popular metrics include:

- Token-based metrics:
  - WER (Word Error Rate)
  - BLEU (BiLingual Evaluation Understudy)
  - TER (Translation Error Rate)
  - METEOR (Metric for Evaluation of Translation with Explicit ORdering)
  - chrF++ (Character F-score)
- Model-based metrics:
  - COMET (Cross-lingual Optimized Metric for Evaluation of Translation)

BLEU: Motivation

- N-gram precision between the machine translation and the reference
- Compute n-gram precisions for 1 to n (typically $n = 4$)
- A geometric mean of the n-gram precisions is calculated over different n's
- Compute on the entire test set, rather than single sentences, to avoid 0 value for higher n-gram precisions

BLEU: Definition

$$\text{BLEU} = \underbrace{\min\left(1,\ e^{1-\frac{r}{c}}\right)}_{\text{Brevity Penalty}} \times \underbrace{\left(\prod_{i=1}^{n} \text{Precision}_i\right)^{\frac{1}{n}}}_{\text{N-gram Precision}}$$

$$\text{Precision}_i = \frac{\mathit{clipped_number_of_matched_\{i\}grams_in_MT}}{\mathit{number_of_total_\{i\}grams_in_MT}}$$

- Brevity Penalty (c — length of MT, r — length of reference) is used to avoid that a very short MT gets a high BLEU score.
- The number of matched $\{i\}$grams is *clipped* to avoid that a single word in the reference is matched multiple times in MT.

Multiple References

To avoid translation variability, multiple references can be used:

- n-grams may match in any of the references
- closest reference length is used for brevity penalty

BLEU: An example

- **Candidate 1: the book is on the desk**

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$$\sum_{unigram \in C} Count(unigram) = 6$$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$$\sum_{unigram \in C} Count(unigram) = 6$$

$$p_1 = 1$$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$$\sum_{unigram \in C} Count(unigram) = 6$$

$$p_1 = 1$$

bigram:

$Count_{clip}(the, book) = 1$

$Count_{clip}(book, is) = 1$

$Count_{clip}(is, on) = 1$

$Count_{clip}(on, the) = 1$

$Count_{clip}(the, desk) = 1$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$$\sum_{unigram \in C} Count(unigram) = 6$$

$$p_1 = 1$$

bigram:

$Count_{clip}(the, book) = 1$

$Count_{clip}(book, is) = 1$

$Count_{clip}(is, on) = 1$

$Count_{clip}(on, the) = 1$

$Count_{clip}(the, desk) = 1$

$$\sum_{bigram \in C} Count(bigram) = 5$$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$\sum_{unigram \in C} Count(unigram) = 6$

$p_1 = 1$

bigram:

$Count_{clip}(the, book) = 1$

$Count_{clip}(book, is) = 1$

$Count_{clip}(is, on) = 1$

$Count_{clip}(on, the) = 1$

$Count_{clip}(the, desk) = 1$

$\sum_{bigram \in C} Count(bigram) = 5$

$p_2 = 1$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:
$Count_{clip}(the) = 2$
$Count_{clip}(book) = 1$
$Count_{clip}(is) = 1$
$Count_{clip}(on) = 1$
$Count_{clip}(desk) = 1$
$\sum_{unigram \in C} Count(unigram) = 6$
$p_1 = 1$

bigram:
$Count_{clip}(the, book) = 1$
$Count_{clip}(book, is) = 1$
$Count_{clip}(is, on) = 1$
$Count_{clip}(on, the) = 1$
$Count_{clip}(the, desk) = 1$
$\sum_{bigram \in C} Count(bigram) = 5$
$p_2 = 1$

trigram:
$Count_{clip}(the, book, is) = 1$
$Count_{clip}(book, is, on) = 1$
$Count_{clip}(is, on, the) = 1$
$Count_{clip}(on, the, desk) = 1$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$\sum_{unigram \in C} Count(unigram) = 6$

$p_1 = 1$

bigram:

$Count_{clip}(the, book) = 1$

$Count_{clip}(book, is) = 1$

$Count_{clip}(is, on) = 1$

$Count_{clip}(on, the) = 1$

$Count_{clip}(the, desk) = 1$

$\sum_{bigram \in C} Count(bigram) = 5$

$p_2 = 1$

trigram:

$Count_{clip}(the, book, is) = 1$

$Count_{clip}(book, is, on) = 1$

$Count_{clip}(is, on, the) = 1$

$Count_{clip}(on, the, desk) = 1$

$\sum_{trigram \in C} Count(trigram) = 4$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$\sum_{unigram \in C} Count(unigram) = 6$

$p_1 = 1$

bigram:

$Count_{clip}(the, book) = 1$

$Count_{clip}(book, is) = 1$

$Count_{clip}(is, on) = 1$

$Count_{clip}(on, the) = 1$

$Count_{clip}(the, desk) = 1$

$\sum_{bigram \in C} Count(bigram) = 5$

$p_2 = 1$

trigram:

$Count_{clip}(the, book, is) = 1$

$Count_{clip}(book, is, on) = 1$

$Count_{clip}(is, on, the) = 1$

$Count_{clip}(on, the, desk) = 1$

$\sum_{trigram \in C} Count(trigram) = 4$

$p_3 = 1$

BLEU: An example

- **Candidate 1: the book is on the desk**

- **Reference 1: there is a book on the desk**
- **Reference 2: the book is on the table**

unigram:

$Count_{clip}(the) = 2$

$Count_{clip}(book) = 1$

$Count_{clip}(is) = 1$

$Count_{clip}(on) = 1$

$Count_{clip}(desk) = 1$

$\sum_{unigram \in C} Count(unigram) = 6$

$p_1 = 1$

bigram:

$Count_{clip}(the, book) = 1$

$Count_{clip}(book, is) = 1$

$Count_{clip}(is, on) = 1$

$Count_{clip}(on, the) = 1$

$Count_{clip}(the, desk) = 1$

$\sum_{bigram \in C} Count(bigram) = 5$

$p_2 = 1$

trigram:

$Count_{clip}(the, book, is) = 1$

$Count_{clip}(book, is, on) = 1$

$Count_{clip}(is, on, the) = 1$

$Count_{clip}(on, the, desk) = 1$

$\sum_{trigram \in C} Count(trigram) = 4$

$p_3 = 1$

$$\left.\begin{matrix} c = 6 \\ r = 6 \end{matrix}\right\} = e^{1-\frac{r}{c}} = e^0 = 1 = BP$$

$$BLEU = BP \times (p_1 \times p_2 \times p_3)^{\frac{1}{3}}$$
$$= 1 \times (1 \times 1 \times 1)^{\frac{1}{3}} = 1$$

Content

1 Machine Translation (MT)

2 Machine translation evaluation

3 Statistical machine translation (SMT)

4 Neural machine translation (NMT) based on RNNs

5 Neural machine translation (NMT) with attention

Content

3 Statistical machine translation (SMT)
- SMT: basic ideas
- Word-based Translation Models

Motivation

- In the time of rule-based MT, the translation knowledge was explicitly encoded by linguists, which is labor intensive, time consuming and inefficient.
- The basic idea of statistical machine translation is to learn a machine translation system directly from parallel translation data, without involving linguistics to encode translation knowledge.

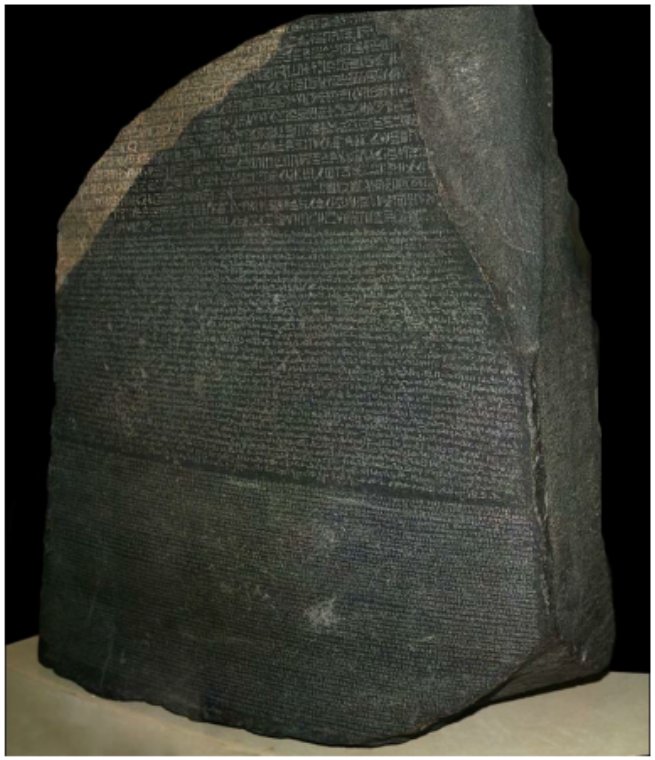
A brief history of SMT

- The first statistical machine translation was conducted by IBM researchers in late 1980s to early 1990s.
- The research of SMT was not very active in 1990s because the computing resources used by IBM researchers were far beyond what most researchers could afford.
- In 1999, a group of researchers gathered in the JHU summer workshop and repeated IBM's early work successfully and released GIZA++, the implementation of the training algorithm of IBM models.
- After 2000 SMT became a dominant MT paradigm until it was replaced by NMT in the mid-2010s.

Parallel corpus

- A parallel corpus, or a bitext, is a collection of texts in one language and its corresponding translation in another language.
- Parallel corpora used for MT research are normally aligned at sentence level.
- Document-level MT needs parallel corpora aligned at word, sentence and document level.
- Many parallel corpora are aligned only in document level in their original form, so a technique called sentence alignment was developed to align the sentences in a document aligned corpus.

Rosetta Stone



	The Rosetta Stone
Material	Granodiorite
Size	114.4 × 72.3 × 27.93 cm (45 x 28.5 x 11 in)
Writing	Ancient Egyptian hieroglyphs, Demotic script, and Greek script
Created	196 BC
Discovered	1799
Present	British Museum

The Rosetta Stone is a good example that it is feasible to learn translation knowledge from a parallel corpus.

Photo by Hans Hillewaert, CC BY-SA 4.0, from Wikimedia Commons; infobox from Wikipedia, “Rosetta Stone”

Statistical Translation Model

A statistical translation model is the probability that a target text e is the translation of a given source text f:

$$p(e|f), \text{where} \sum_{e} p(e|f) = 1$$

Statistical Machine Translation

Given a statistical translation model $p(e|f)$, the machine translation problem can be transformed to a search problem: to search a target sentence e which has the highest translation probability given f:

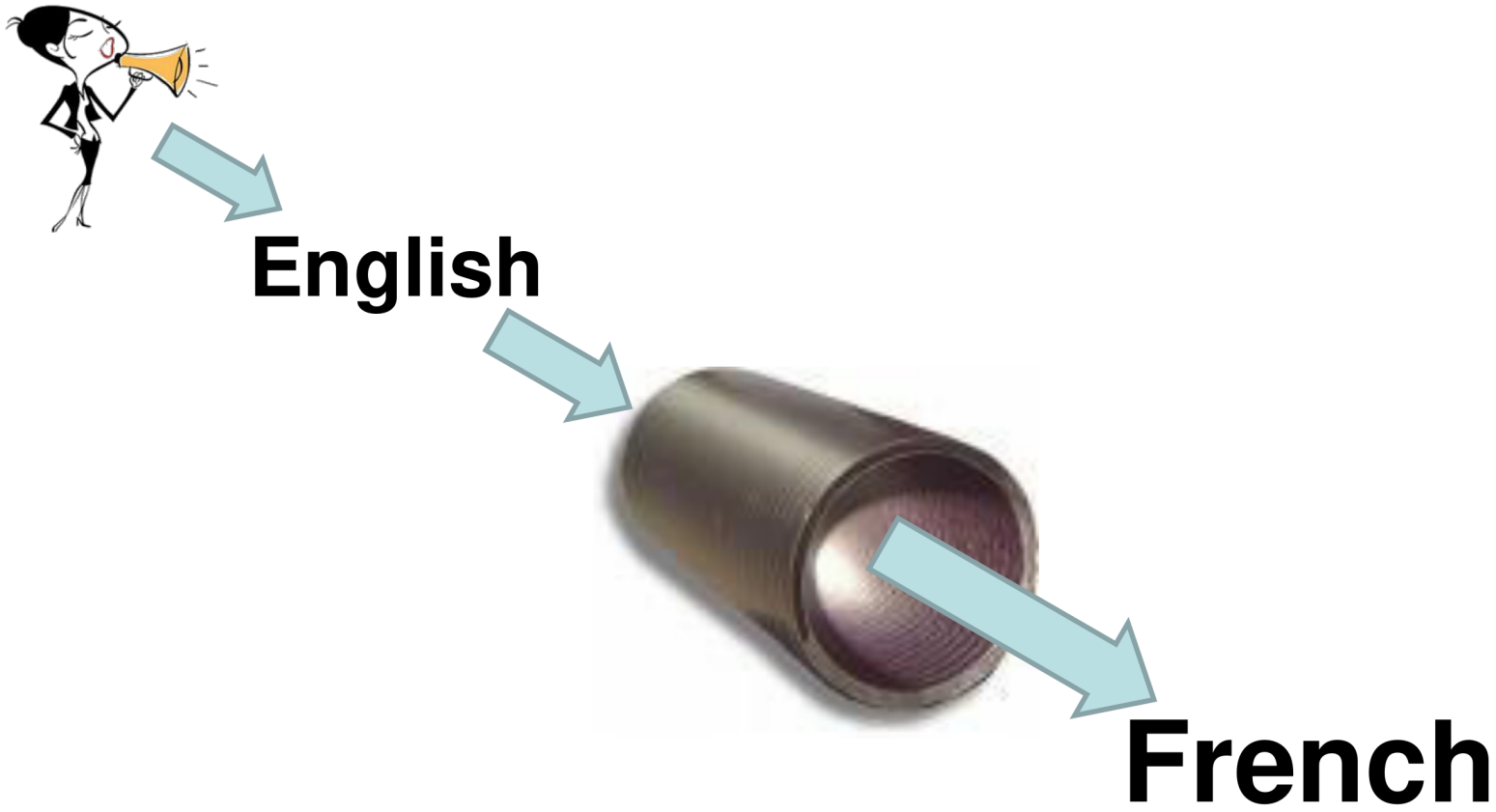
$$\hat{e} = \underset{e}{\operatorname{argmax}}\, p(e|f)$$

Noisy Channel Model

- Statistical machine translation with a single translation model does not work well.
- IBM researchers proposed a Noisy Channel Model for statistical machine translation, which includes an additional language model.

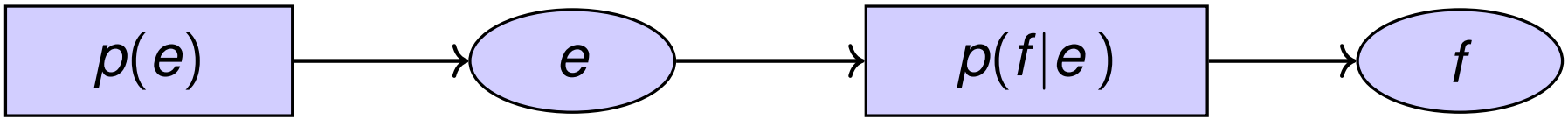
> Peter F. Brown, John Cocke, Stephen A. Della Pietra, Vincent J. Della Pietra, Frederick Jelinek, John D. Lafferty, Robert L. Mercer, Paul S. Roossin, A Statistical Approach to Machine Translation, Computational Linguistics, 1990

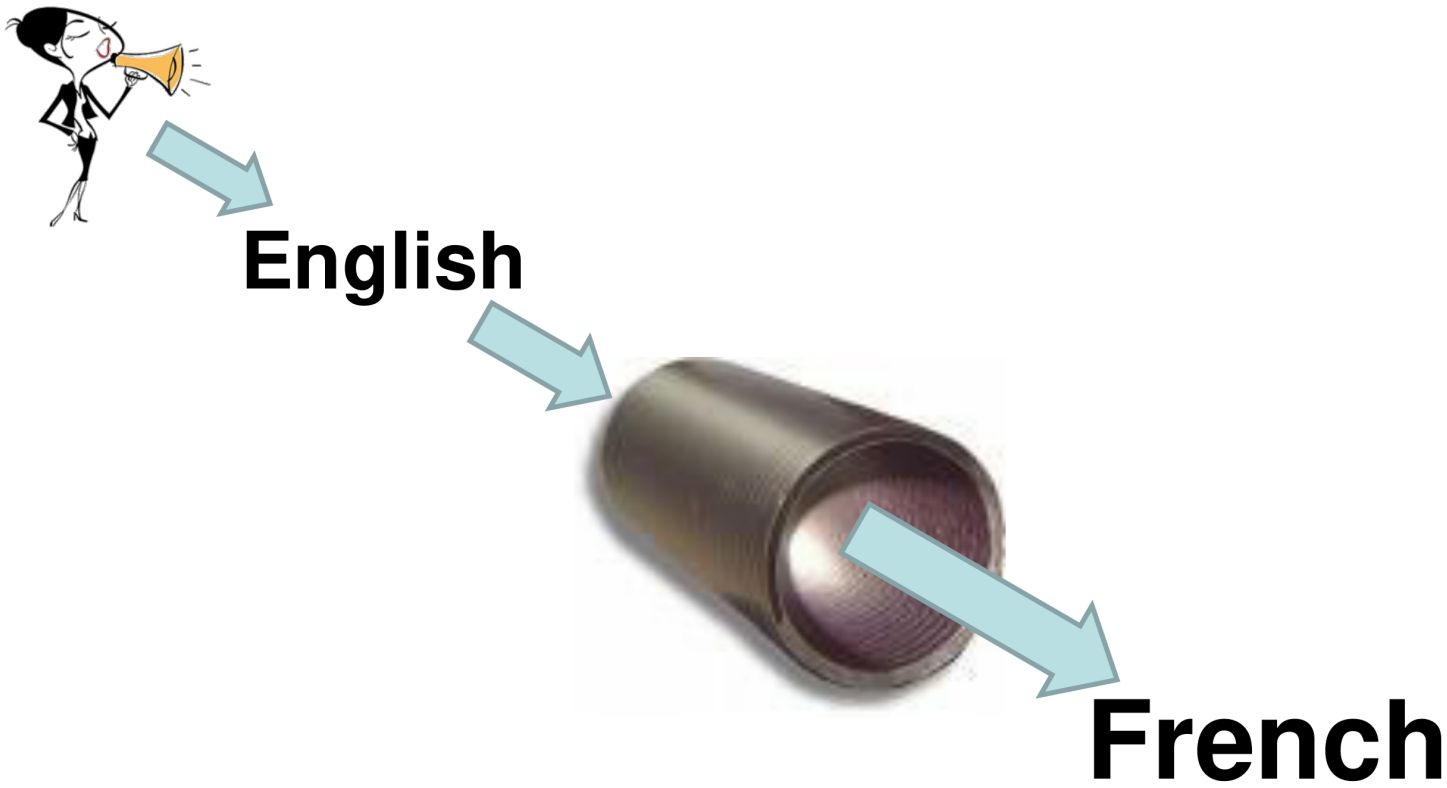
Noisy Channel Framework



A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

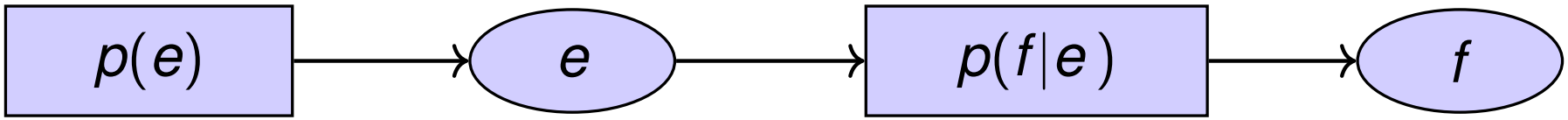
Noisy Channel Framework





A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Noisy Channel Framework



$$p(f) = p(e)\,p(f|e)$$



English



French

A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Noisy Channel Framework

Applying Bayes' Rule, we have:

$$p(e|f) = \frac{p(e)\,p(f|e)}{p(f)}$$

Thus:

$$\hat{e} = \underset{e}{\operatorname{argmax}}\, p(e|f) = \underset{e}{\operatorname{argmax}}\, p(e)\,p(f|e)$$

Noisy Channel Framework

Applying Bayes' Rule, we have:

$$p(e|f) = \frac{p(e)\,p(f|e)}{p(f)}$$

Thus:

$$\hat{e} = \operatorname*{argmax}_e p(e|f) = \operatorname*{argmax}_e p(e)\,p(f|e)$$

Noisy Channel Framework

$$\hat{e} = \underbrace{\operatorname*{argmax}_{e}}_{\text{Decoder}} \underbrace{p(e)}_{\text{Language Model}} \underbrace{p(f \mid e)}_{\text{Translation Model}}$$







Figures from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Noisy Channel Framework

- The *translation model* models how likely it is that f is a translation of e – **adequacy**.
- The *language model* models how likely it is that e is an acceptable sentence – **fluency**.
- The *decoder* searches for the most likely e.

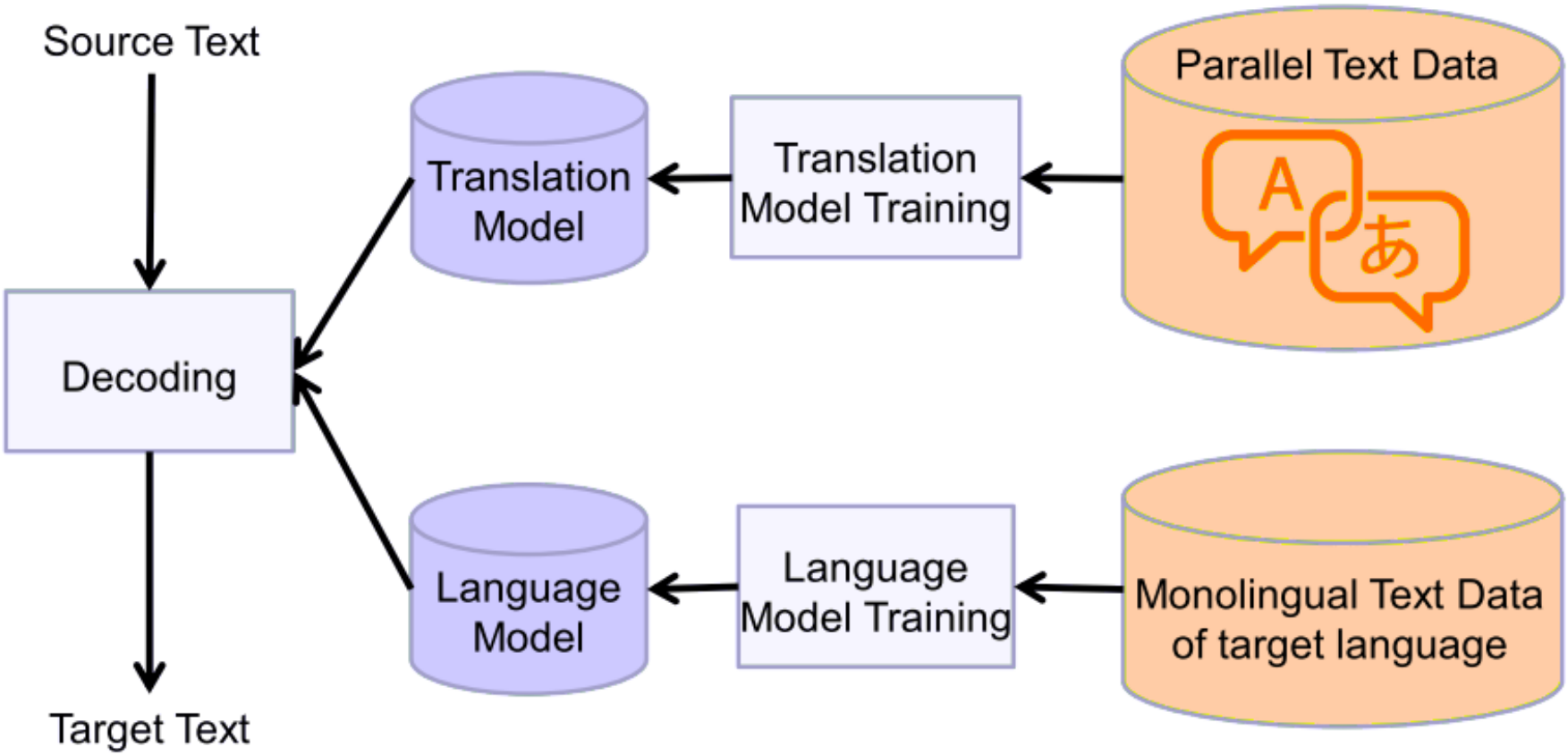
We have introduced language models in previous lectures, here we will mainly focus on translation models.

Noisy Channel Framework

- The *translation model* models how likely it is that f is a translation of e – **adequacy**.
- The *language model* models how likely it is that e is an acceptable sentence – **fluency**.
- The *decoder* searches for the most likely e.

We have introduced language models in previous lectures, here we will mainly focus on translation models.

SMT Workflow



A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Content

3 Statistical machine translation (SMT)
- SMT: basic ideas
- Word-based Translation Models

Categories of translation models

Various translation models have been proposed, which belong to different categories, according to the language units on which they are built up:

- Word-based models
  - IBM models 1-5
  - HMM models
- Phrase-based models
- Syntax-based models
  - Tree-to-string models
  - String-to-tree models
  - Tree-to-tree models
  - Dependency-based models

IBM Models

IBM researchers proposed 5 models with increased complexity:

- IBM Model 1: only consider lexical translation probabilities
- IBM Model 2: add an absolute reordering model
- IBM Model 3: add a fertility model
- IBM Model 4: add a relative reordering model
- IBM Model 5: improve the previous model with more parameters

Lexical translation probabilities

German	English	Prob.
das	the	0.7
das	that	0.15
das	which	0.075
das	who	0.05
das	this	0.025
...	...	...

German	English	Prob.
Haus	house	0.8
Haus	building	0.16
Haus	home	0.02
Haus	household	0.015
Haus	shell	0.005
...	...	...

Word alignment

- To estimate the word translation probabilities, we need alignment between words in the parallel sentences



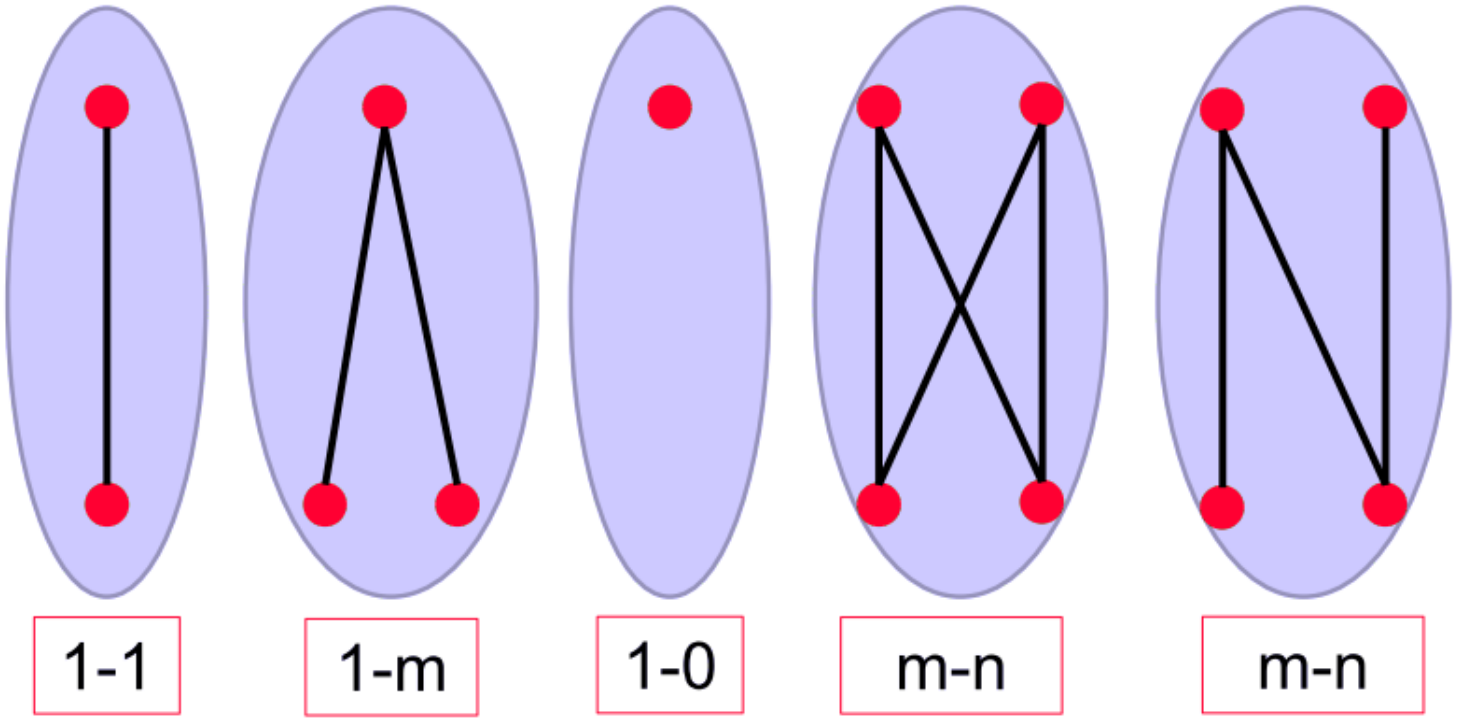






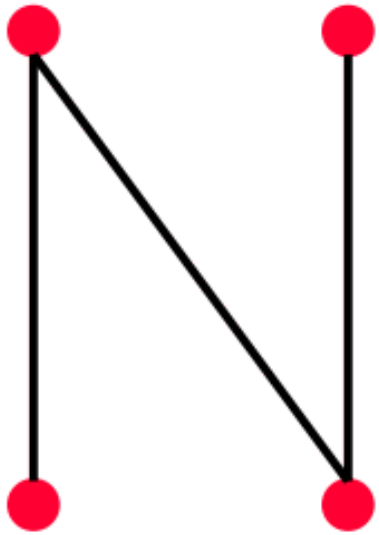
Examples from Philipp Koehn, Statistical Machine Translation, Cambridge University Press, 2010

Word alignment patterns



A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

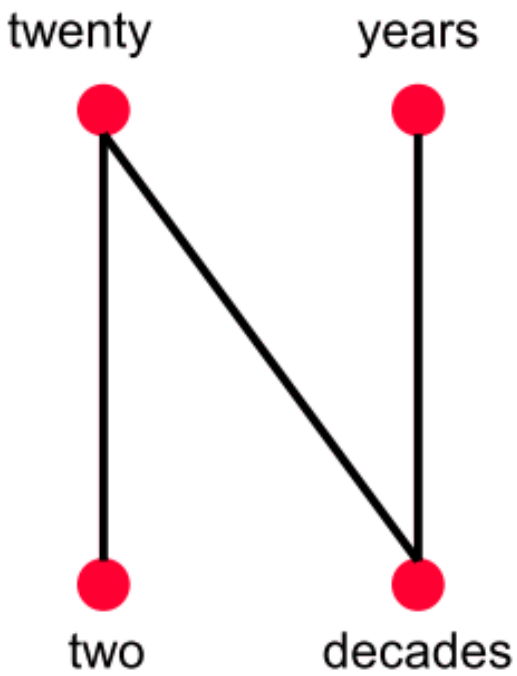
Word alignment patterns



Can you imagine a word alignment pattern like this?

A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Word alignment patterns



Can you imagine a word alignment pattern like this?

A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Learning lexical translation models

- We would like to estimate the lexical translation probabilities from a parallel corpus...
- but we do not have the alignments:
  - If we had the alignments, we could estimate the lexical translation probabilities.
  - If we had the probabilities, we could estimate the alignments.

Learning lexical translation models

- We would like to estimate the lexical translation probabilities from a parallel corpus...
- but we do not have the alignments:
  - If we had the alignments, we could estimate the lexical translation probabilities.
  - If we had the probabilities, we could estimate the alignments.

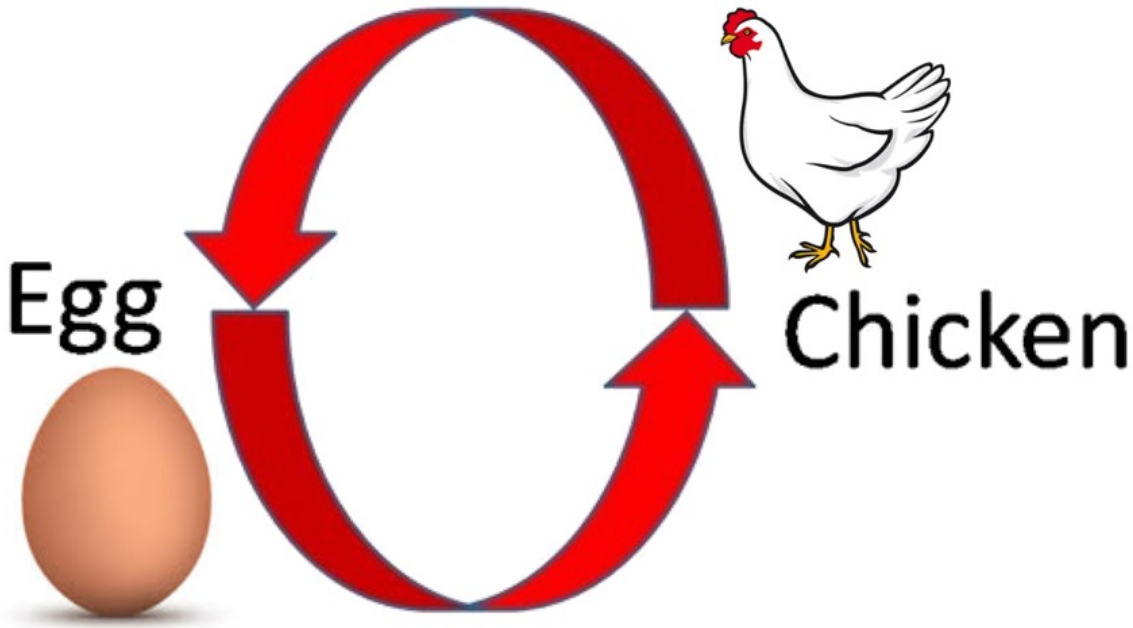
Learning lexical translation models

- We would like to estimate the lexical translation probabilities from a parallel corpus...
- but we do not have the alignments:
  - If we had the alignments, we could estimate the lexical translation probabilities.
  - If we had the probabilities, we could estimate the alignments.

Learning lexical translation models

- We would like to estimate the lexical translation probabilities from a parallel corpus...
- but we do not have the alignments:
  - If we had the alignments, we could estimate the lexical translation probabilities.
  - If we had the probabilities, we could estimate the alignments.

A Paradox



A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

EM Algorithm

- Incomplete data
  - If we had complete data, we could estimate the model.
  - If we had the model, we could fill in the gaps in the data.
- Solution: Expectation Maximization (EM) Algorithm
  - Initialize model parameters. (e.g. uniform)
  - Assign probabilities to the missing data. (E-step)
  - Estimate model parameters from completed data. (M-step)
  - Iterate E-step and M-step until the model converges.

How does EM algorithm work?

EM Algorithm consists of two steps:

1. Expectation-Step: Apply model to the data
   - parts of the data are hidden (here: alignments)
   - using the model, assign probabilities of the hidden data to possible values (alignments)
2. Maximization-Step: Estimate new model from data
   - take assigned values as fact
   - collect counts (weighted by probabilities)
   - estimate new model from counts

Iterate the E-step and the M-step until convergence

Example

Consider a parallel corpus containing just two pairs:

blue house	maison bleue
house	maison

- How many possible alignments in the first pair?
- How many in the second pair?
- We will simplify the example by ruling out many-to-one or zero-to-one alignments.

Example

Consider a parallel corpus containing just two pairs:

blue house	maison bleue
house	maison

- How many possible alignments in the first pair?
- How many in the second pair?
- We will simplify the example by ruling out many-to-one or zero-to-one alignments.

Example

Consider a parallel corpus containing just two pairs:

blue house	maison bleue
house	maison

- How many possible alignments in the first pair? 2
- How many in the second pair? 1
- We will simplify the example by ruling out many-to-one or zero-to-one alignments.

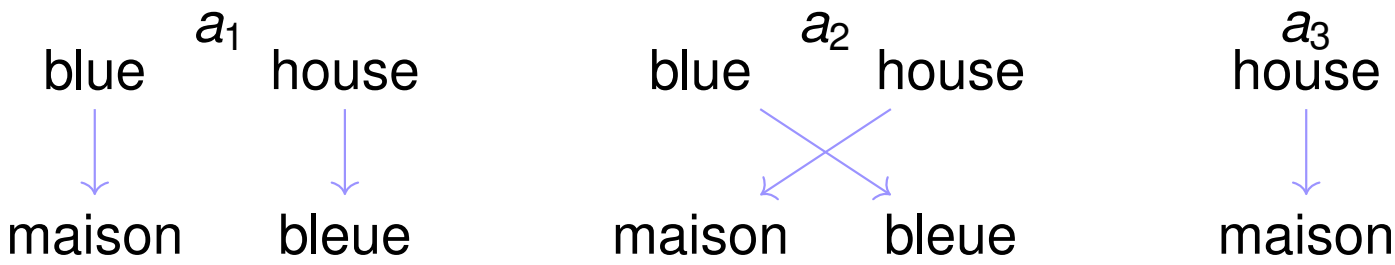
Step 1 (Initialisation)

Set parameter values uniformly.

- $t(\text{bleue}|\text{house}) = 1/2$
- $t(\text{maison}|\text{house}) = 1/2$
- $t(\text{bleue}|\text{blue}) = 1/2$
- $t(\text{maison}|\text{blue}) = 1/2$

Steps 2–3 (Expectation)

Compute the probability of all alignments:



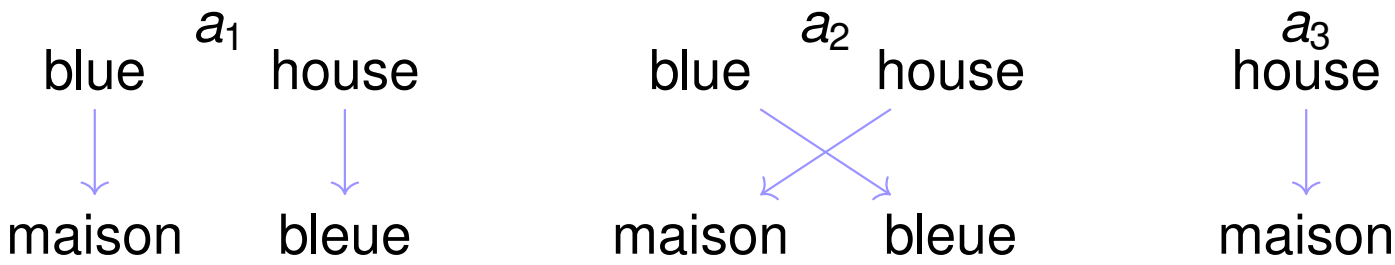
- $p(a_1, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{blue}) \cdot t(\text{bleue}|\text{house}) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$
- $p(a_2, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{house}) \cdot t(\text{bleue}|\text{blue}) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$
- $p(a_3, \text{maison}|\text{house}) = t(\text{maison}|\text{house}) = \frac{1}{2}$

Normalise for all alignments:

- $p(a_1|\text{maison bleue, blue house}) = 1/4 \div 2/4 = 1/2$
- $p(a_2|\text{maison bleue, blue house}) = 1/4 \div 2/4 = 1/2$
- $p(a_3|\text{maison, house}) = 1/2 \div 1/2 = 1$

Steps 2–3 (Expectation)

Compute the probability of all alignments:



- $p(a_1, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{blue}) \cdot t(\text{bleue}|\text{house}) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$
- $p(a_2, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{house}) \cdot t(\text{bleue}|\text{blue}) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$
- $p(a_3, \text{maison}|\text{house}) = t(\text{maison}|\text{house}) = \frac{1}{2}$

Normalise for all alignments:

- $p(a_1|\text{maison bleue}, \text{blue house}) = 1/4 \div 2/4 = 1/2$
- $p(a_2|\text{maison bleue}, \text{blue house}) = 1/4 \div 2/4 = 1/2$
- $p(a_3|\text{maison}, \text{house}) = 1/2 \div 1/2 = 1$

Steps 4–5 (Maximisation)

Collect fractional counts:

- $tc(\text{bleue}|\text{house}) = 1/2$
- $tc(\text{maison}|\text{house}) = 1/2 + 1 = 3/2$
- $tc(\text{bleue}|\text{blue}) = 1/2$
- $tc(\text{maison}|\text{blue}) = 1/2$

Normalise fractional counts to yield revised parameter values:

- $t(\text{bleue}|\text{house}) = 1/2 \div 2 = 1/4$
- $t(\text{maison}|\text{house}) = 3/2 \div 2 = 3/4$
- $t(\text{bleue}|\text{blue}) = 1/2 \div 1 = 1/2$
- $t(\text{maison}|\text{blue}) = 1/2 \div 1 = 1/2$

Steps 4–5 (Maximisation)

Collect fractional counts:

- $tc(\text{bleue}|\text{house}) = 1/2$
- $tc(\text{maison}|\text{house}) = 1/2 + 1 = 3/2$
- $tc(\text{bleue}|\text{blue}) = 1/2$
- $tc(\text{maison}|\text{blue}) = 1/2$

Normalise fractional counts to yield revised parameter values:

- $t(\text{bleue}|\text{house}) = 1/2 \div 2 = 1/4$
- $t(\text{maison}|\text{house}) = 3/2 \div 2 = 3/4$
- $t(\text{bleue}|\text{blue}) = 1/2 \div 1 = 1/2$
- $t(\text{maison}|\text{blue}) = 1/2 \div 1 = 1/2$

Repeat Steps 2–3 (Expectation)

Compute the probability of all alignments:

- $p(a_1, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{blue}) \cdot t(\text{bleue}|\text{house}) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8}$
- $p(a_2, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{house}) \cdot t(\text{bleue}|\text{blue}) = \frac{3}{4} \cdot \frac{1}{2} = \frac{3}{8}$
- $p(a_3, \text{maison}|\text{house}) = t(\text{maison}|\text{house}) = \frac{3}{4}$

Normalise for all alignments:

- $p(a_1|\text{maison bleue, blue house}) = 1/8 \div 4/8 = 1/4$
- $p(a_2|\text{maison bleue, blue house}) = 3/8 \div 4/8 = 3/4$
- $p(a_3|\text{maison, house}) = 3/4 \div 3/4 = 1$

Repeat Steps 2–3 (Expectation)

Compute the probability of all alignments:

- $p(a_1, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{blue}) \cdot t(\text{bleue}|\text{house}) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8}$
- $p(a_2, \text{maison bleue}|\text{blue house}) = t(\text{maison}|\text{house}) \cdot t(\text{bleue}|\text{blue}) = \frac{3}{4} \cdot \frac{1}{2} = \frac{3}{8}$
- $p(a_3, \text{maison}|\text{house}) = t(\text{maison}|\text{house}) = \frac{3}{4}$

Normalise for all alignments:

- $p(a_1|\text{maison bleue, blue house}) = 1/8 \div 4/8 = 1/4$
- $p(a_2|\text{maison bleue, blue house}) = 3/8 \div 4/8 = 3/4$
- $p(a_3|\text{maison, house}) = 3/4 \div 3/4 = 1$

Repeat Steps 4–5 (Maximisation)

Collect fractional counts:

- $tc(\text{bleue}|\text{house}) = 1/4$
- $tc(\text{maison}|\text{house}) = 3/4 + 1 = 7/4$
- $tc(\text{bleue}|\text{blue}) = 3/4$
- $tc(\text{maison}|\text{blue}) = 1/4$

Normalise fractional counts to yield revised parameter values:

- $t(\text{bleue}|\text{house}) = 1/4 \div 2 = 1/8$
- $t(\text{maison}|\text{house}) = 7/4 \div 2 = 7/8$
- $t(\text{bleue}|\text{blue}) = 3/4 \div 1 = 3/4$
- $t(\text{maison}|\text{blue}) = 1/4 \div 1 = 1/4$

Repeat Steps 4–5 (Maximisation)

Collect fractional counts:

- $tc(\text{bleue}|\text{house}) = 1/4$
- $tc(\text{maison}|\text{house}) = 3/4 + 1 = 7/4$
- $tc(\text{bleue}|\text{blue}) = 3/4$
- $tc(\text{maison}|\text{blue}) = 1/4$

Normalise fractional counts to yield revised parameter values:

- $t(\text{bleue}|\text{house}) = 1/4 \div 2 = 1/8$
- $t(\text{maison}|\text{house}) = 7/4 \div 2 = 7/8$
- $t(\text{bleue}|\text{blue}) = 3/4 \div 1 = 3/4$
- $t(\text{maison}|\text{blue}) = 1/4 \div 1 = 1/4$

Convergence

Repeating steps 2, 3, 4 and 5 eventually yields:

- $t(\text{bleue}|\text{house}) = 0.0001$
- $t(\text{maison}|\text{house}) = 0.9999$
- $t(\text{bleue}|\text{blue}) = 0.9999$
- $t(\text{maison}|\text{blue}) = 0.0001$

EM is guaranteed to converge, but in general only to a *local* optimum (for IBM Model 1 the likelihood is concave, so it reaches the global one).

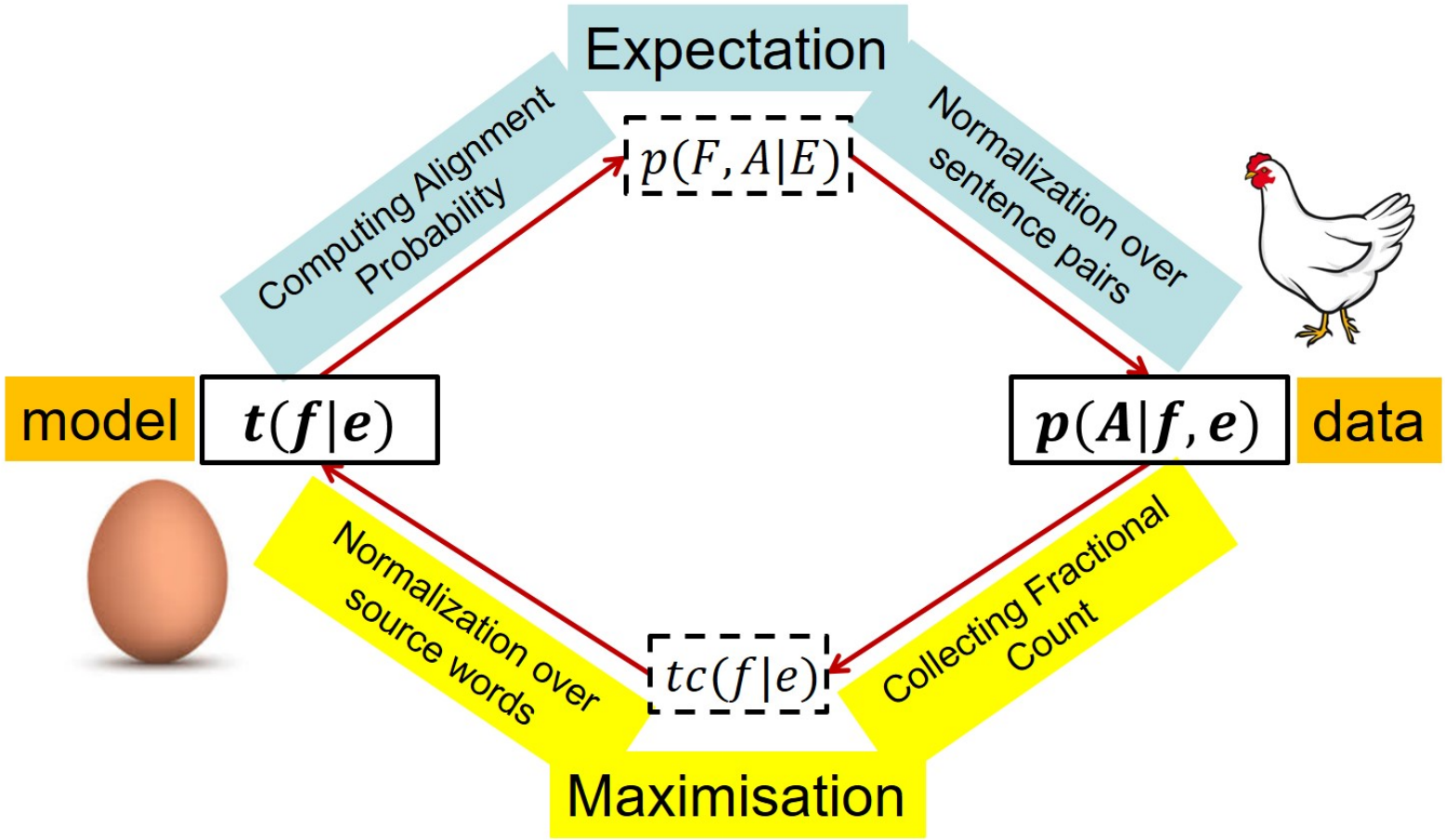
Convergence

Repeating steps 2, 3, 4 and 5 eventually yields:

- $t(\text{bleue}|\text{house}) = 0.0001$
- $t(\text{maison}|\text{house}) = 0.9999$
- $t(\text{bleue}|\text{blue}) = 0.9999$
- $t(\text{maison}|\text{blue}) = 0.0001$

EM is guaranteed to converge, but in general only to a *local* optimum (for IBM Model 1 the likelihood is concave, so it reaches the global one).

EM Algorithm



A figure from Qun Liu, CA446 Statistical Machine Translation, Dublin City University

Content

1 Machine Translation (MT)

2 Machine translation evaluation

3 Statistical machine translation (SMT)

4 Neural machine translation (NMT) based on RNNs

5 Neural machine translation (NMT) with attention

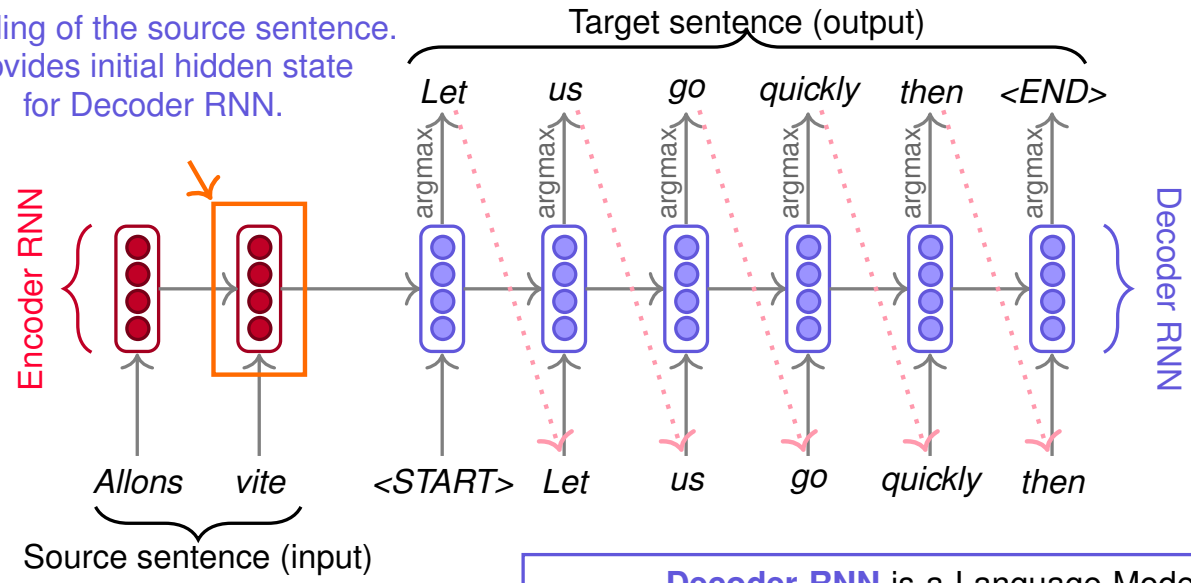
What is Neural Machine Translation?

- Neural Machine Translation (NMT) is a way to do Machine Translation with a **single neural network**
- The neural network architecture is called **sequence-to-sequence** (aka seq2seq) and it involves two RNNs.

Neural Machine Translation (NMT)

The sequence-to-sequence model

Encoding of the source sentence. Provides initial hidden state for Decoder RNN.



Encoder RNN produces an encoding of the source sentence.

Decoder RNN is a Language Model that generates target sentence, *conditioned on* encoding.

Note: This diagram shows **test time** behavior: decoder output is fed in ⇢ as next step's input

Sequence-to-sequence is versatile!

- Sequence-to-sequence is useful for more than just MT
- Many NLP tasks can be phrased as sequence-to-sequence:
  - Summarization (long text $\rightarrow$ short text)
  - Dialogue (previous utterances $\rightarrow$ next utterance)
  - Parsing (input text $\rightarrow$ output parse as sequence)
  - Code generation (natural language $\rightarrow$ Python code)

Neural Machine Translation (NMT)

- The sequence-to-sequence model is an example of a **Conditional Language Model**.
  - **Language Model** because the decoder is predicting the next word of the target sentence y
  - **Conditional** because its predictions are also conditioned on the source sentence x
- NMT directly calculates $p(y|x)$:

$$p(y|x) = p(y_1|x)\, p(y_2|y_1, x)\, p(y_3|y_1, y_2, x) \ldots p(y_T|y_1, \ldots, y_{T-1}, x)$$

Probability of next target word, given target words so far and source sentence x

- Question: How to train a NMT system?
- Answer: Get a big parallel corpus...

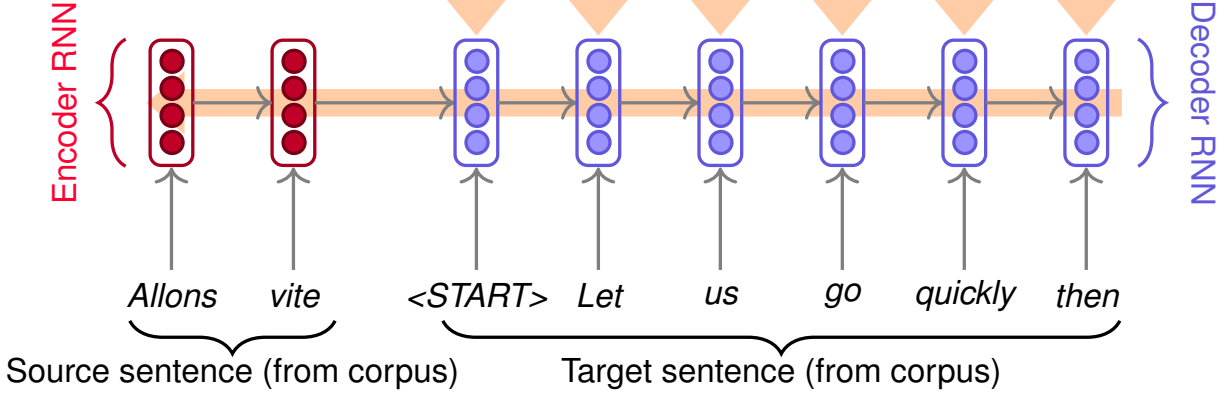
Training a Neural Machine Translation system

$$J = \frac{1}{T}\sum_{t=1}^{T} J_t = J_1 + J_2 + J_3 + J_4 + J_5 + J_6$$

= negative log prob of “Let”

= negative log prob of “go”

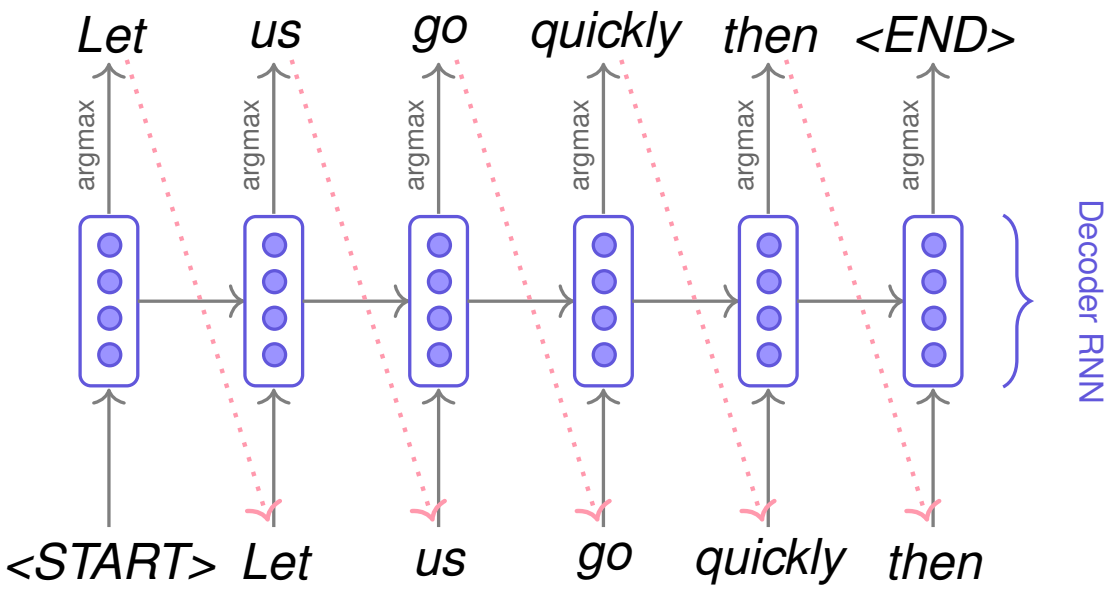
= negative log prob of <END>



Seq2seq is optimized as a **single system.**
Backpropagation operates “*end-to-end*”.

Greedy decoding

- We saw how to generate (or “decode”) the target sentence by taking argmax on each step of the decoder



- This is greedy decoding (take most probable word on each step)
- **Problems with this method?**

Problems with greedy decoding

- Greedy decoding has no way to undo decisions!
- Input: *Allons vite* (Let us go quickly then)

  $\rightarrow$ Let _____

  $\rightarrow$ Let us _____

  $\rightarrow$ Let us hurry _____ (whoops! no going back now. . .)
- How to fix this?

Example: A. Daudet, *Tartarin de Tarascon* (1872); English translation by O. C. Colt (Project Gutenberg)

Exhaustive search decoding

- Ideally we want to find a (length T) translation y that maximizes

$$p(y|x) = p(y_1|x)\, p(y_2|y_1, x)\, p(y_3|y_1, y_2, x) \ldots p(y_T|y_1, \ldots, y_{T-1}, x)$$

$$= \prod_{t=1}^{T} p(y_t|y_1, \ldots, y_{t-1}, x)$$

- We could try computing all possible sequences y
- This means that on each step t of the decoder, we're tracking V^t possible partial translations, where V is vocab size
- This $O(V^T)$ complexity is far too expensive!

Beam search decoding

- Core idea: On each step of decoder, keep track of the k most probable partial translations (which we call *hypotheses*)
  - k is the beam size (in practice around 5 to 10)
- A hypothesis $y_1, \ldots, y_t$ has a *score* which is its log probability:

$$\text{score}(y_1, \ldots, y_t) = \log P_{LM}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{LM}(y_i | y_1, \ldots, y_{i-1}, x)$$

- Scores are all negative, and higher score is better
- We search for high-scoring hypotheses, tracking top k on each step
- Beam search is not guaranteed to find optimal solution
- But much more efficient than exhaustive search!

Beam search decoding: example

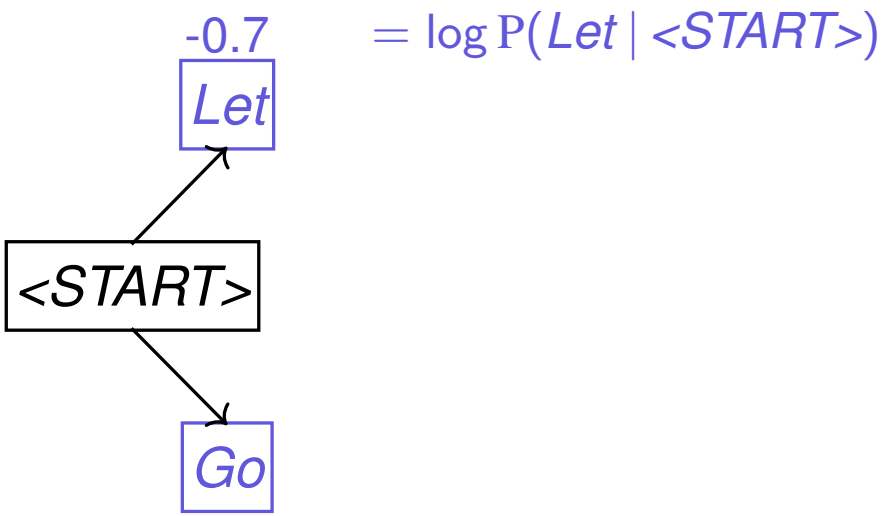
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$

<START>

Calculate prob dist of next word

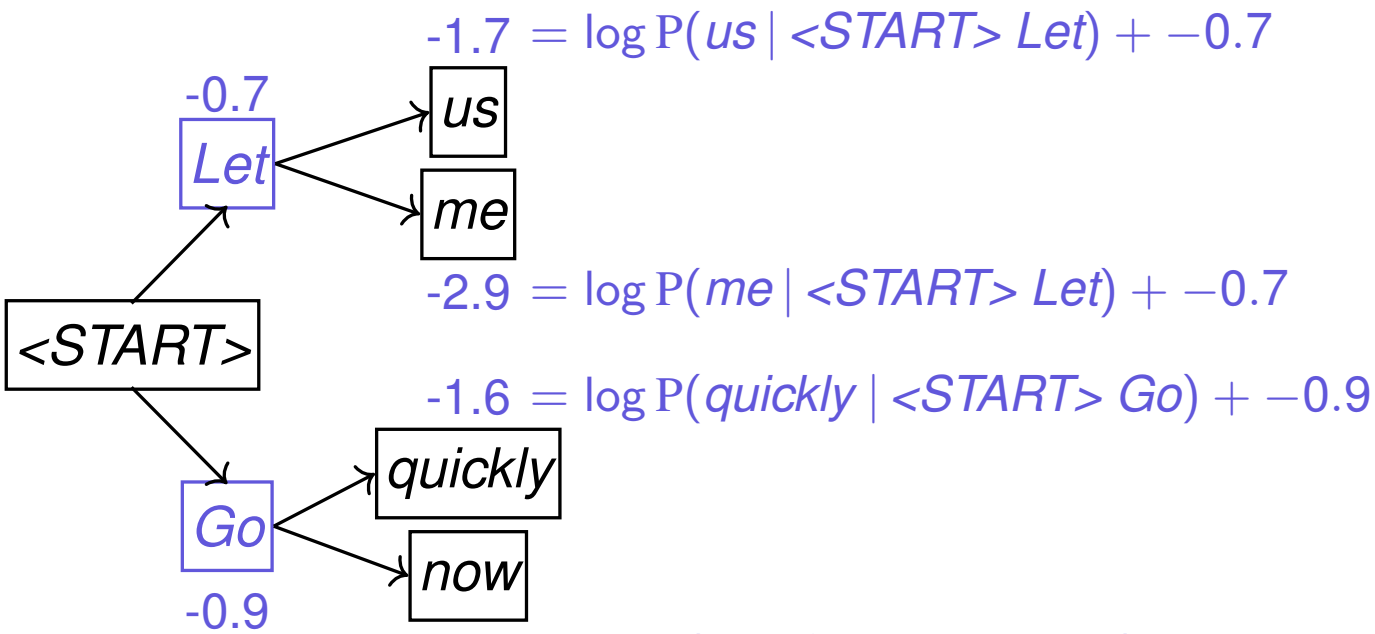
Beam search decoding: example

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



Beam search decoding: example

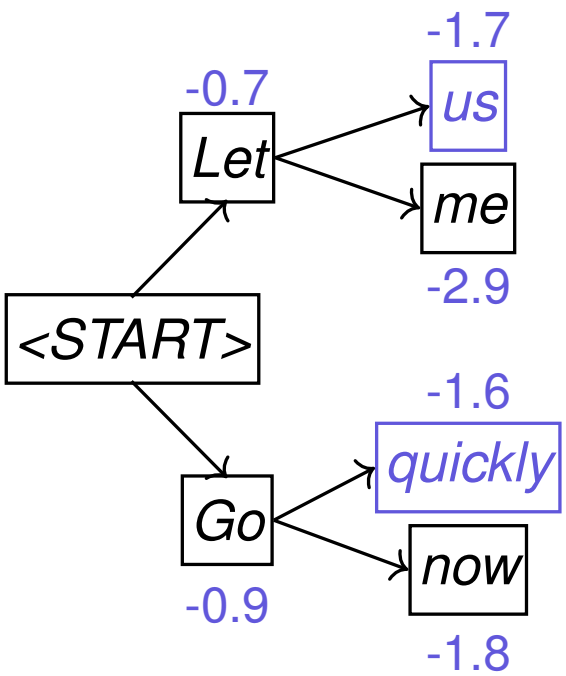
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam search decoding: example

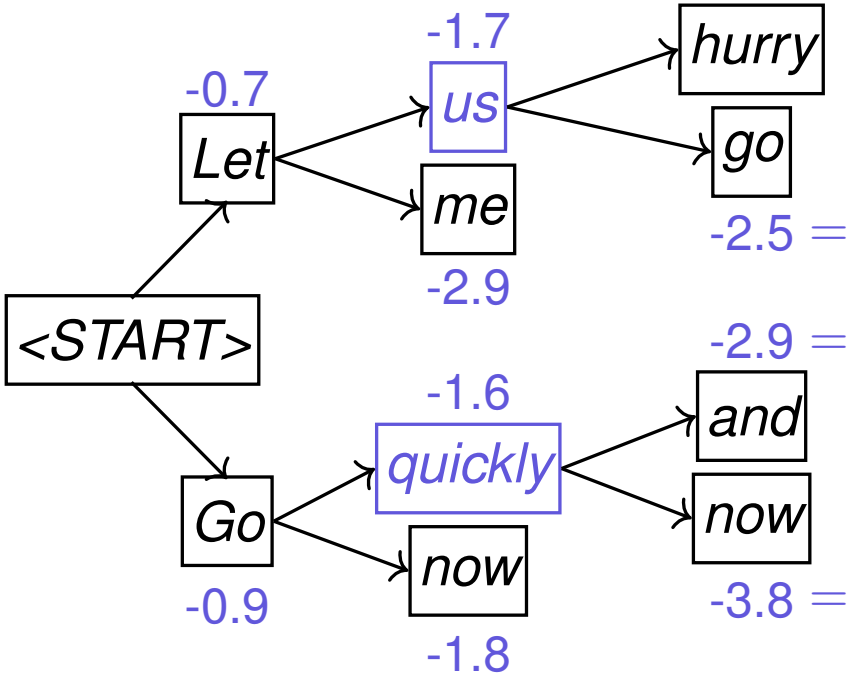
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam search decoding: example

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



-2.8 = log P(*hurry* | <START> *Let us*) + −1.7

-2.5 = log P(*go* | <START> *Let us*) + −1.7

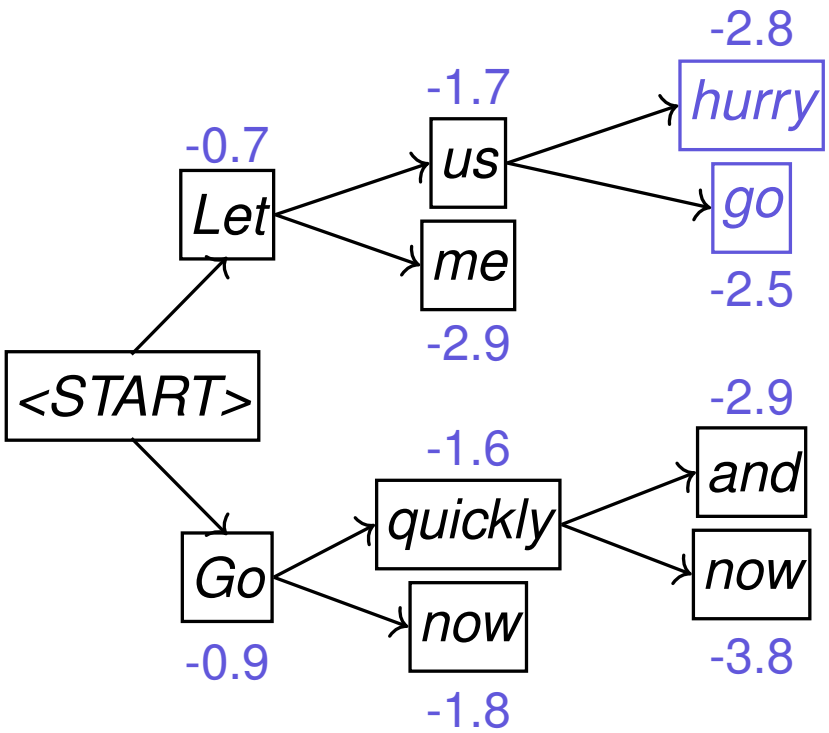
-2.9 = log P(*and* | <START> *Go quickly*) + −1.6

-3.8 = log P(*now* | <START> *Go quickly*) + −1.6

For each of the *k* hypotheses, find top *k* next words and calculate scores

Beam search decoding: example

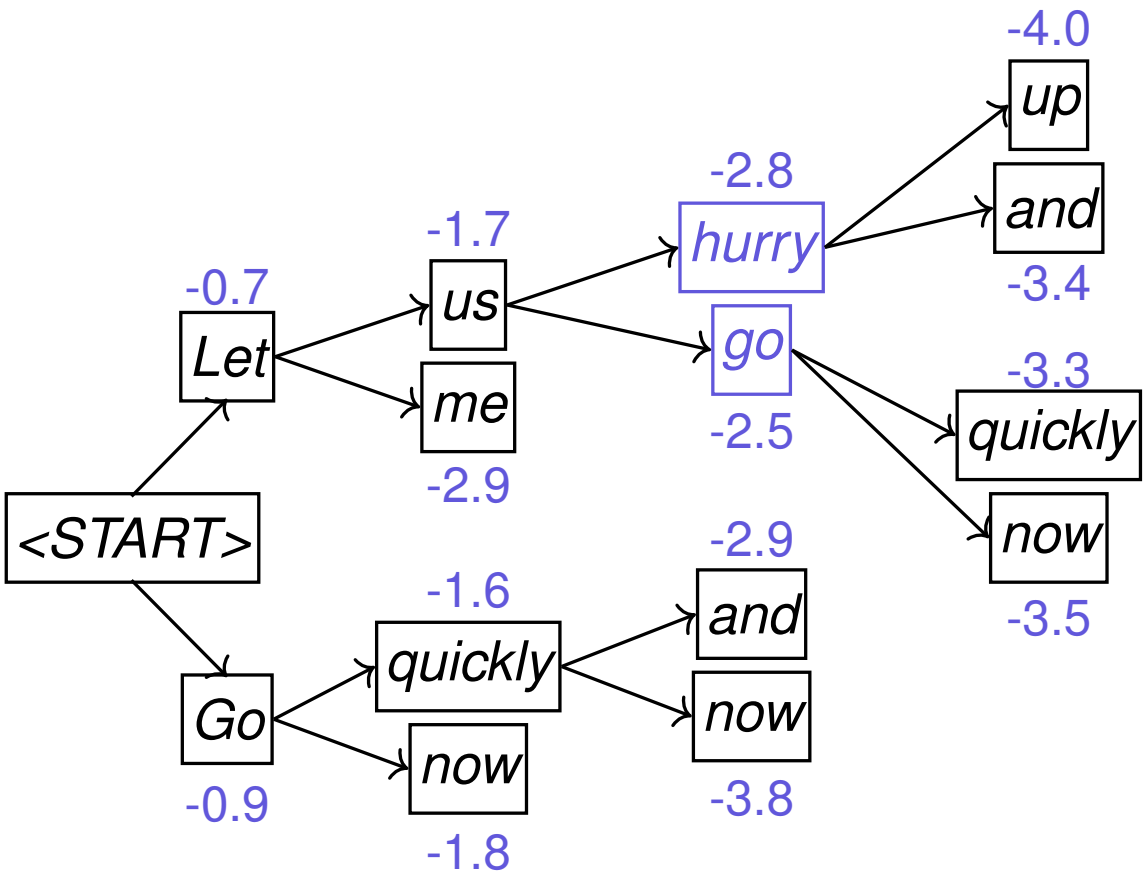
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam search decoding: example

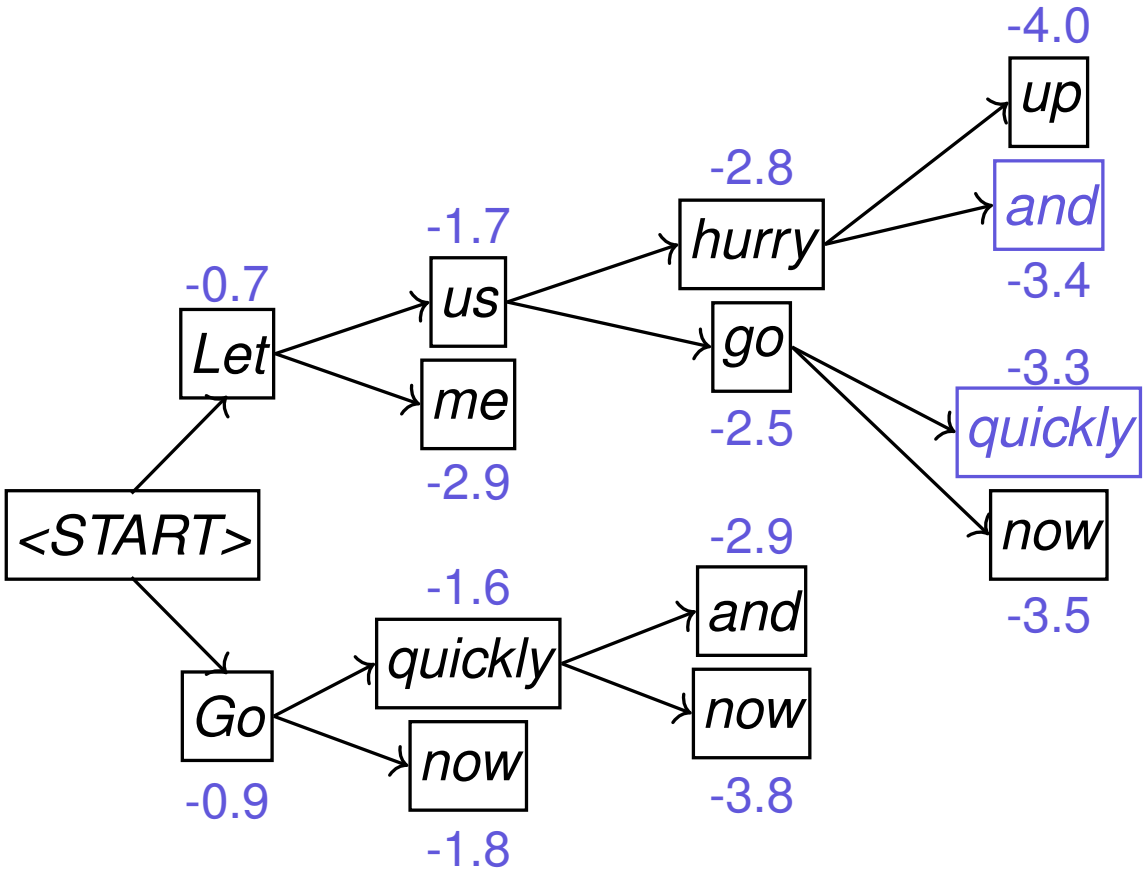
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam search decoding: example

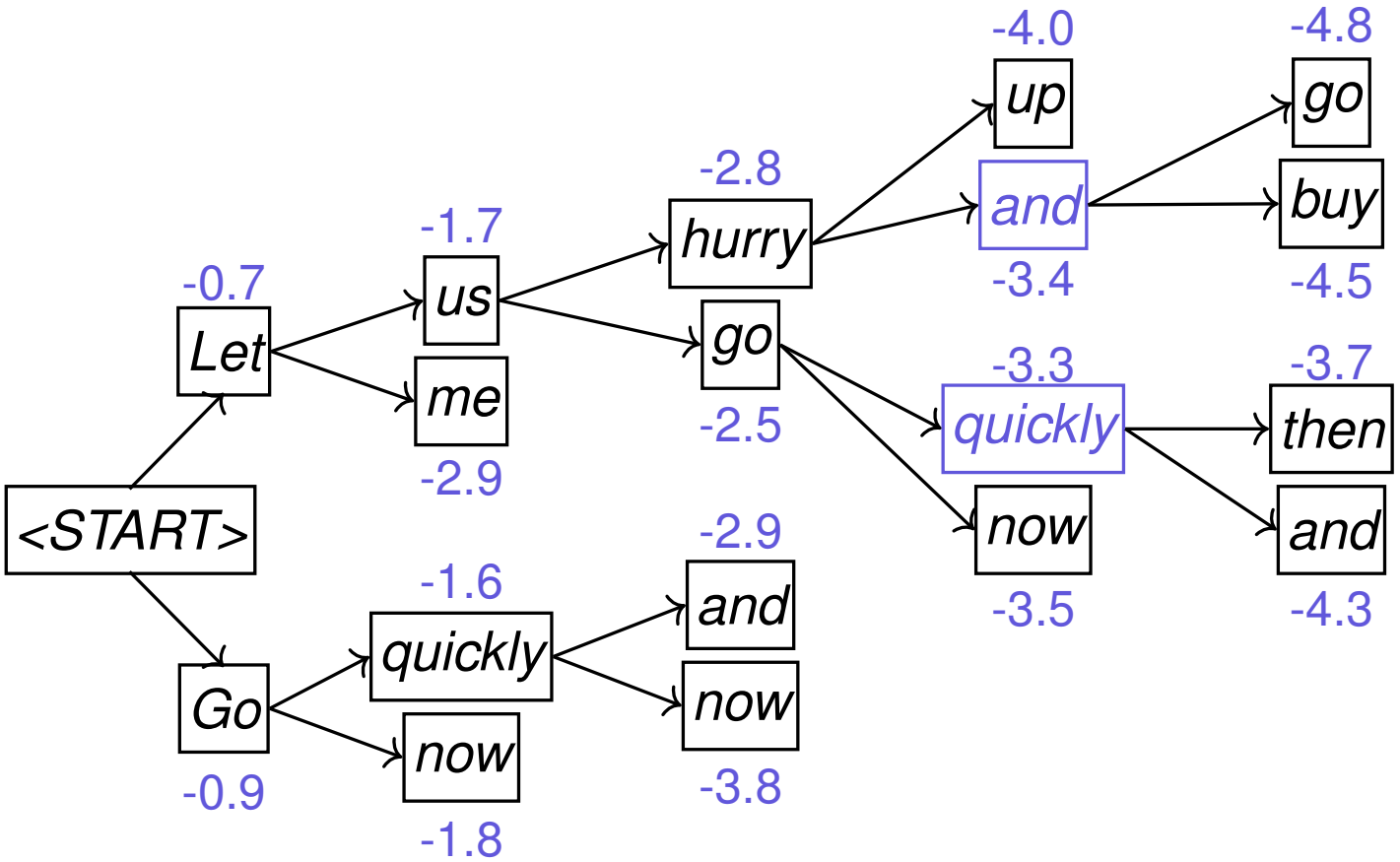
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam search decoding: example

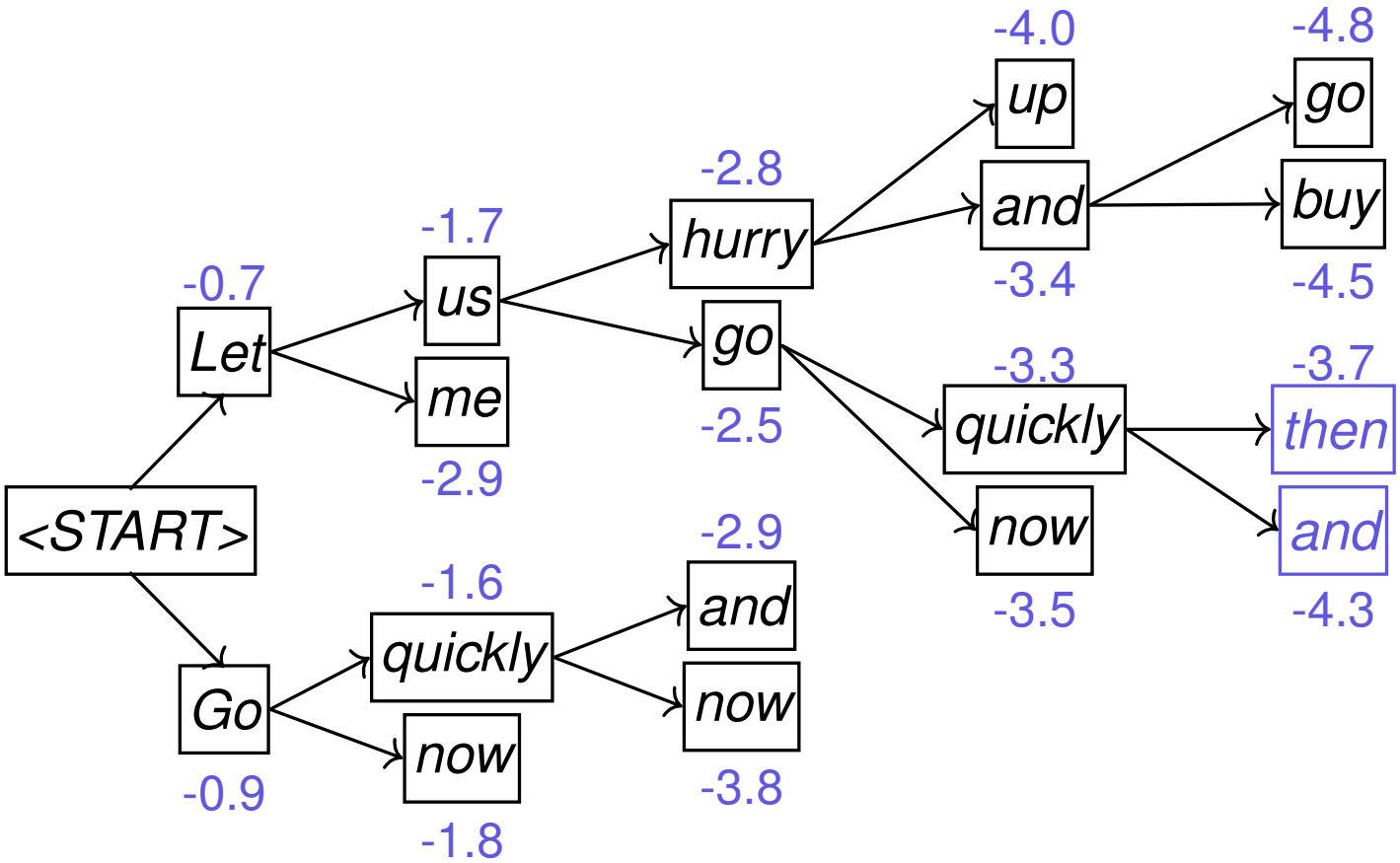
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam search decoding: example

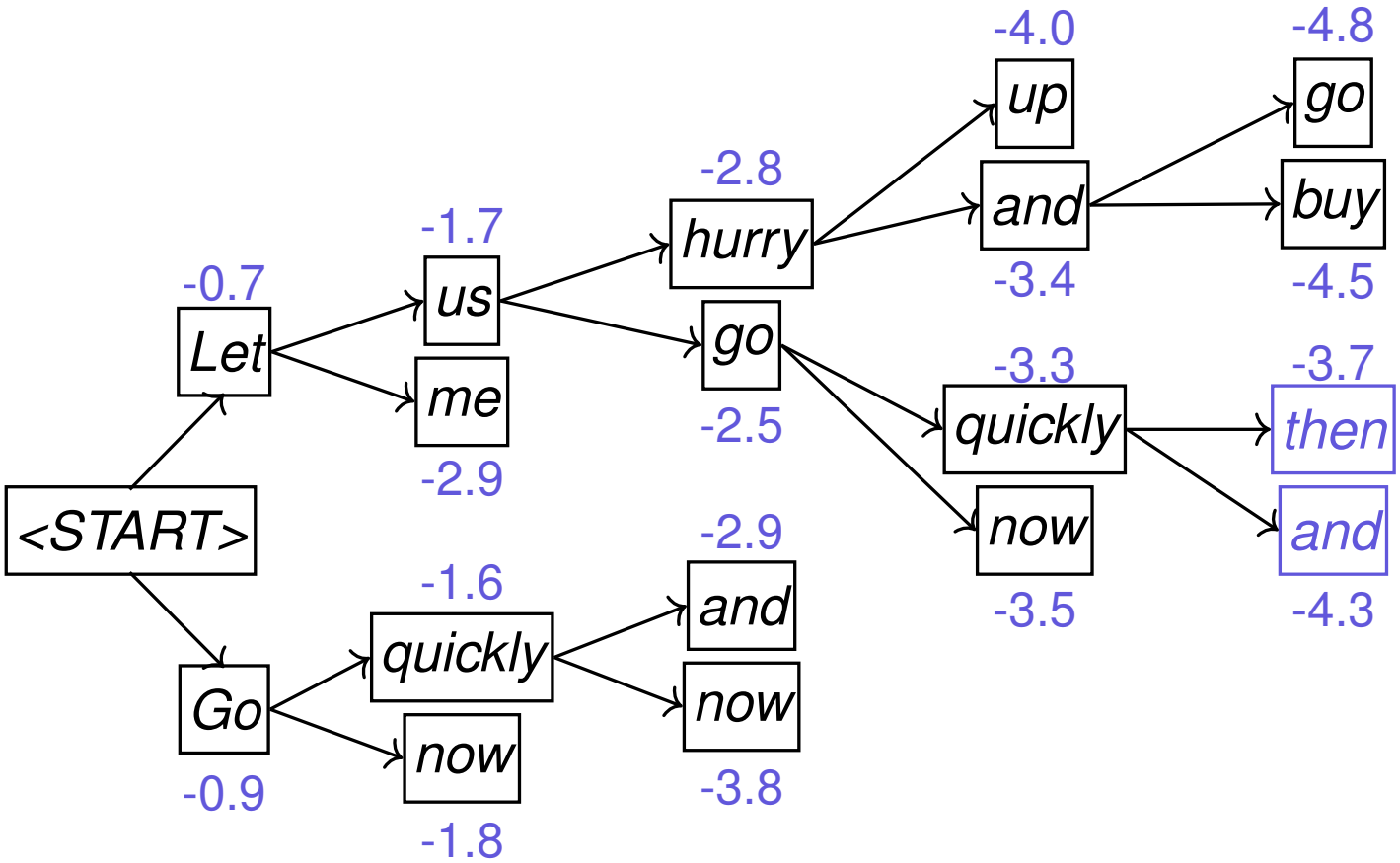
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam search decoding: example

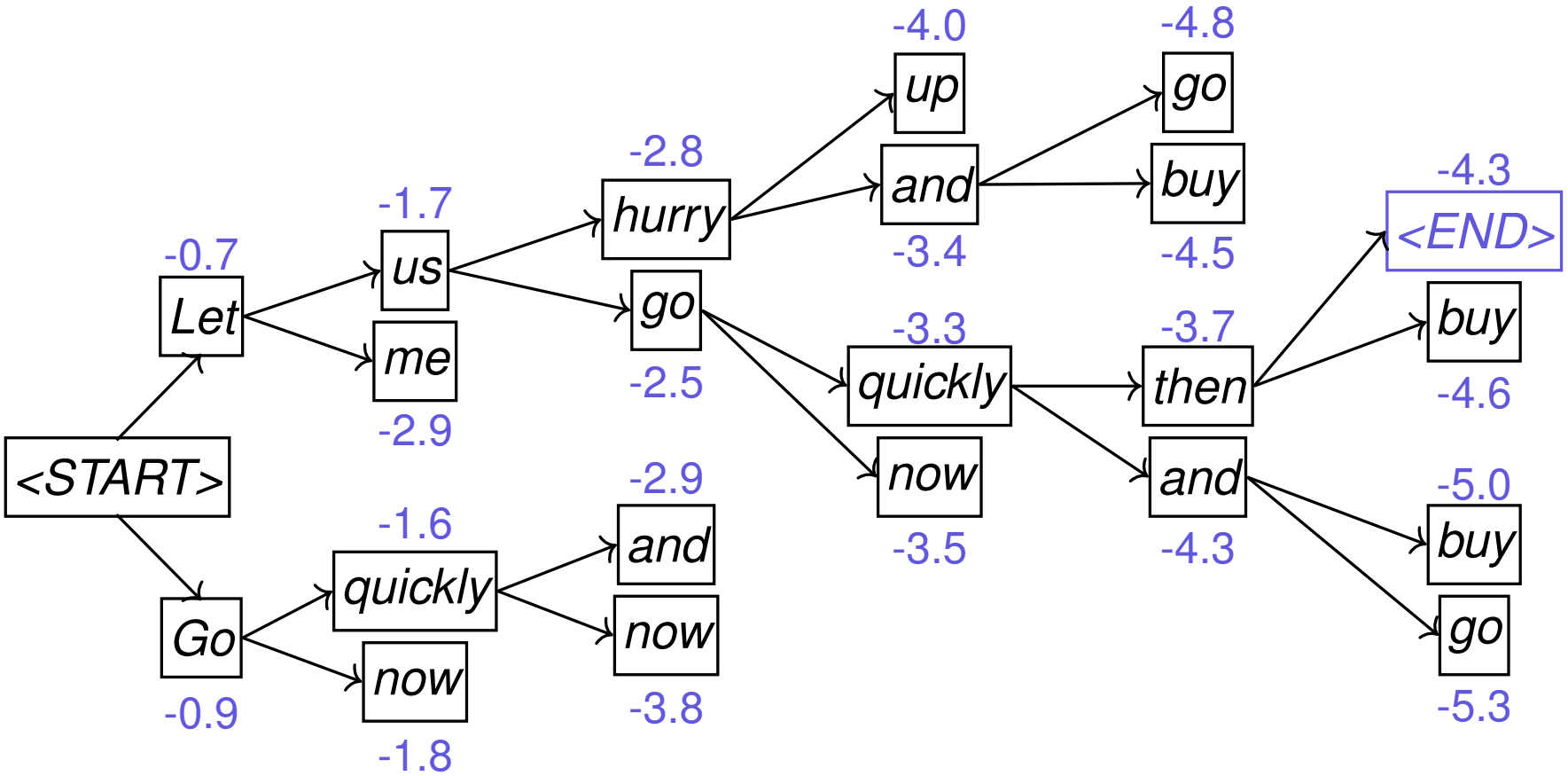
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam search decoding: example

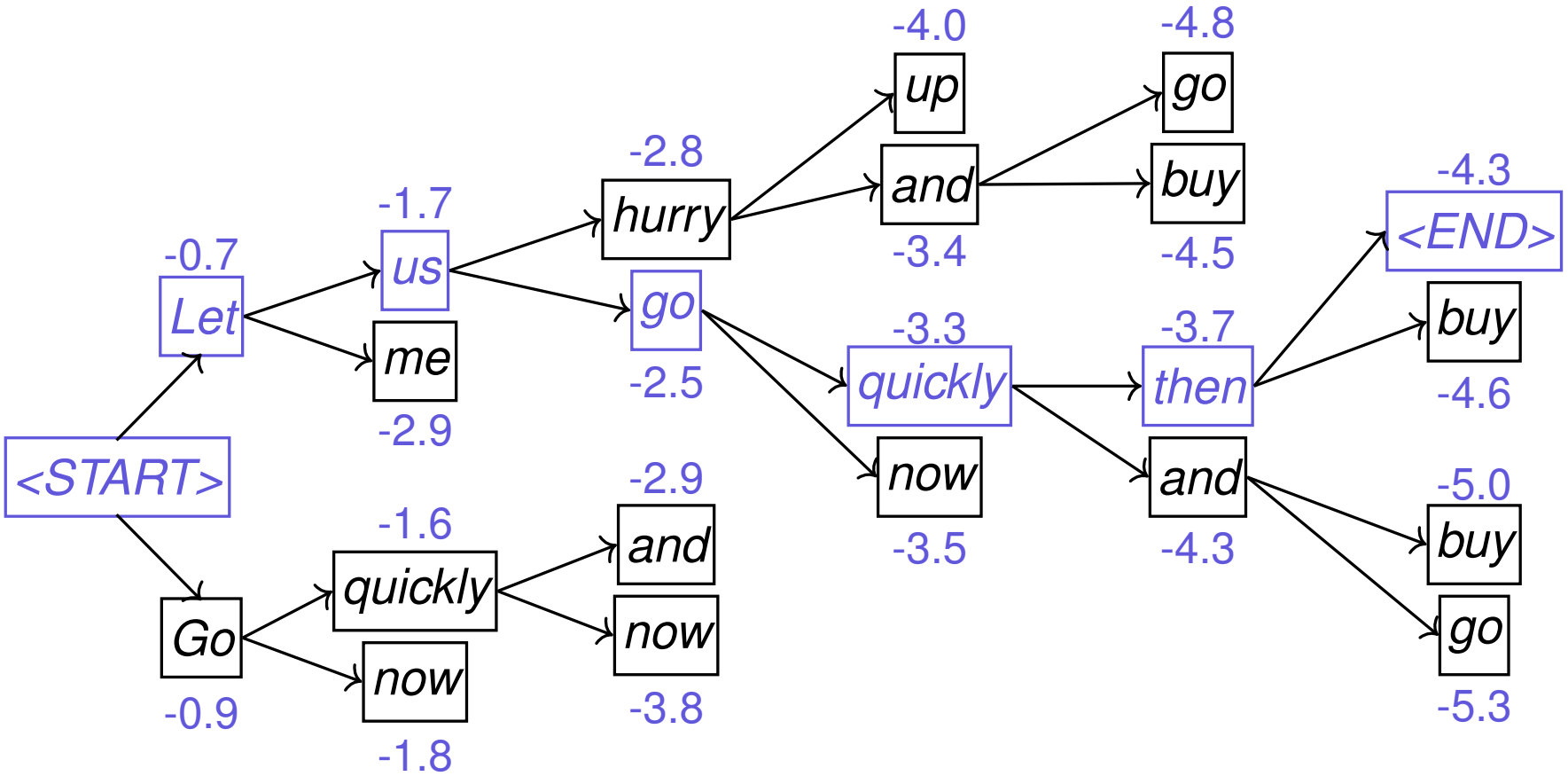
Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



This is the top-scoring hypothesis!

Beam search decoding: example

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i \mid y_1, \ldots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis

Beam search decoding: stopping criterion

- In greedy decoding, usually we decode until the model produces a <END> token
- For example: <START> Let us go quickly then <END>
- In beam search decoding, different hypotheses may produce <END> tokens on different timesteps
- When a hypothesis produces <END>, that hypothesis is complete.
  - Place it aside and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
  - We reach timestep T (where T is some pre-defined cutoff), or
  - We have at least n completed hypotheses (where n is pre-defined cutoff)

Beam search decoding: finishing up

- We have our list of completed hypotheses.
- How to select top one with highest score?
- Each hypothesis $y_1, \ldots, y_t$ on our list has a score

$$\text{score}(y_1, \ldots, y_t) = \log P_{LM}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{LM}(y_i | y_1, \ldots, y_{i-1}, x)$$

- Problem with this: longer hypotheses have lower scores
- Fix: Normalize by length. Use this to select top one instead:

$$\frac{1}{t} \sum_{i=1}^{t} \log P_{LM}(y_i | y_1, \ldots, y_{i-1}, x)$$

Advantages of NMT

Compared to SMT, NMT has many advantages:

- Better performance
  - More fluent
  - Better use of context
  - Better use of phrase similarities
- A single neural network to be optimized end-to-end
  - No subcomponents to be individually optimized
- Requires much less human engineering effort
  - No feature engineering
- Same method for all language pairs

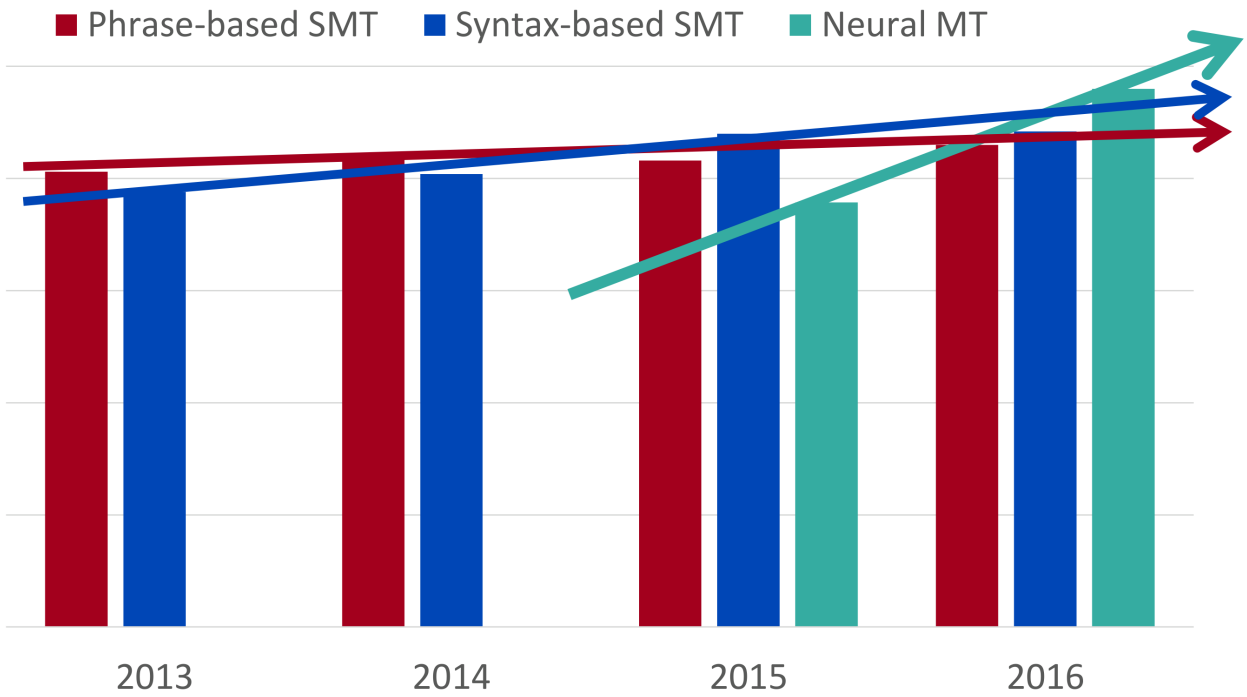
Disadvantages of NMT?

Compared to SMT:

- NMT is less interpretable
  - Hard to debug
- NMT is difficult to control
  - For example, can't easily specify rules or guidelines for translation
- Safety concerns!

MT progress over time

[Edinburgh En-De WMT newstest2013 Cased BLEU; NMT 2015 from U. Montréal]



Source: http://www.meta-net.eu/events/meta-forum-2016/slides/09_sennrich.pdf

NMT: the biggest success story of NLP Deep Learning

Neural Machine Translation went from a fringe research activity in 2014 to the leading standard method in 2016

- 2014: First seq2seq paper published
- 2016: Google Translate switches from SMT to NMT
- This is amazing!
  - SMT systems, built by hundreds of engineers over many years, outperformed by NMT systems trained by a handful of engineers in a few months

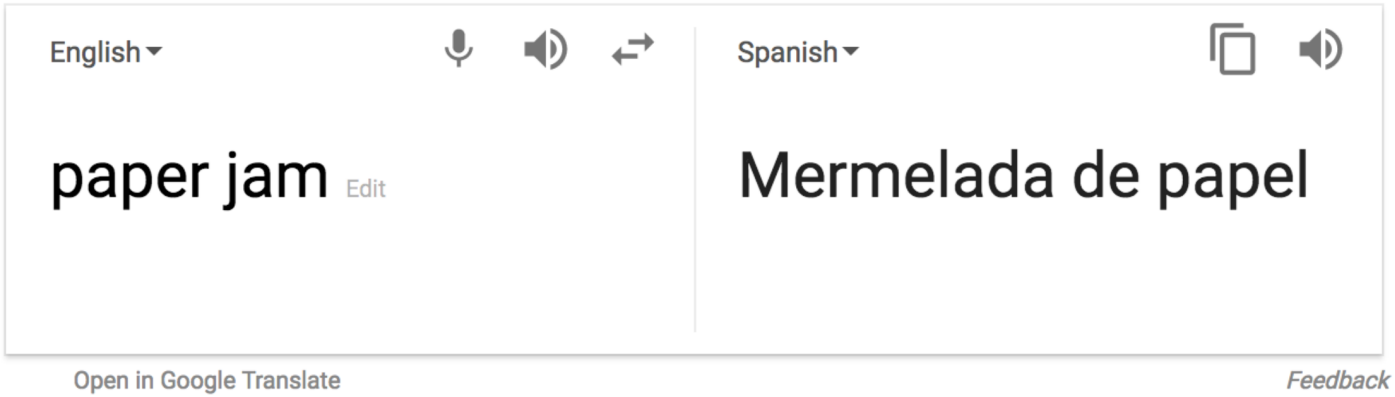
So is Machine Translation solved?

- Nope!
- Many difficulties remain:
  - Out-of-vocabulary words
  - Domain mismatch between train and test data
  - Maintaining context over longer text
  - Low-resource language pairs

Further reading: “Has AI surpassed humans at translation? Not even close!”
`https://www.skynettoday.com/editorials/state_of_nmt`

So is Machine Translation solved?

- **Nope!**
- Using common sense is still hard





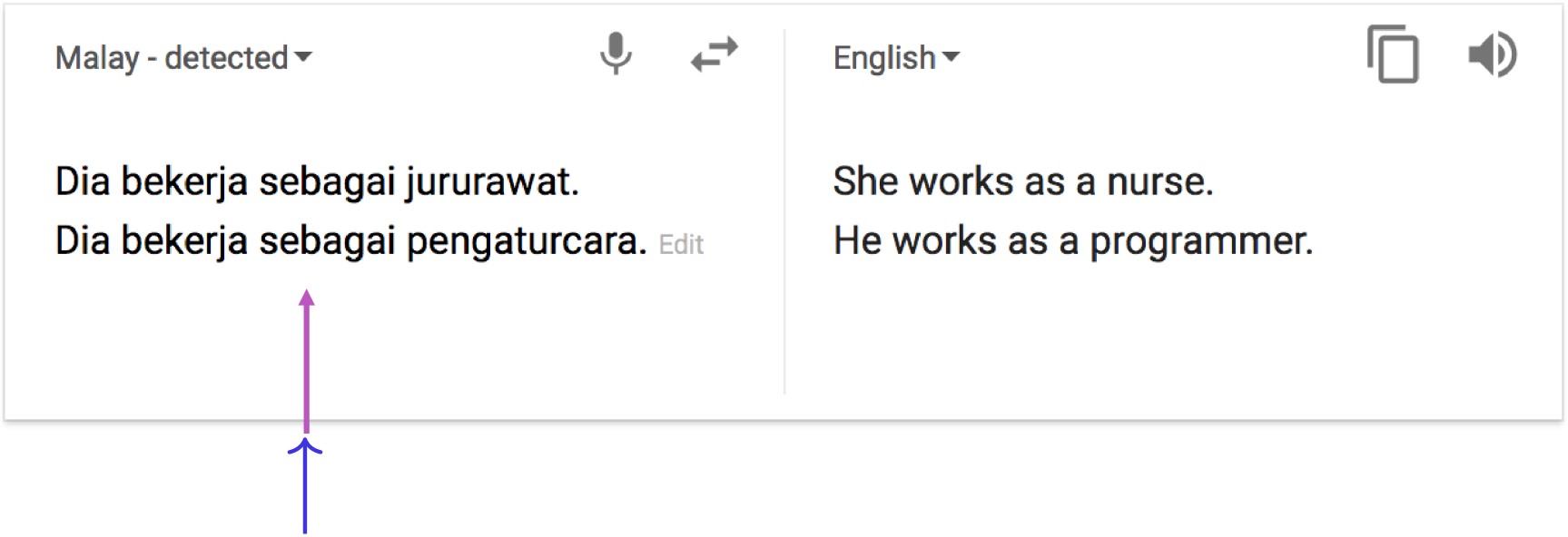


?

A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

So is Machine Translation solved?

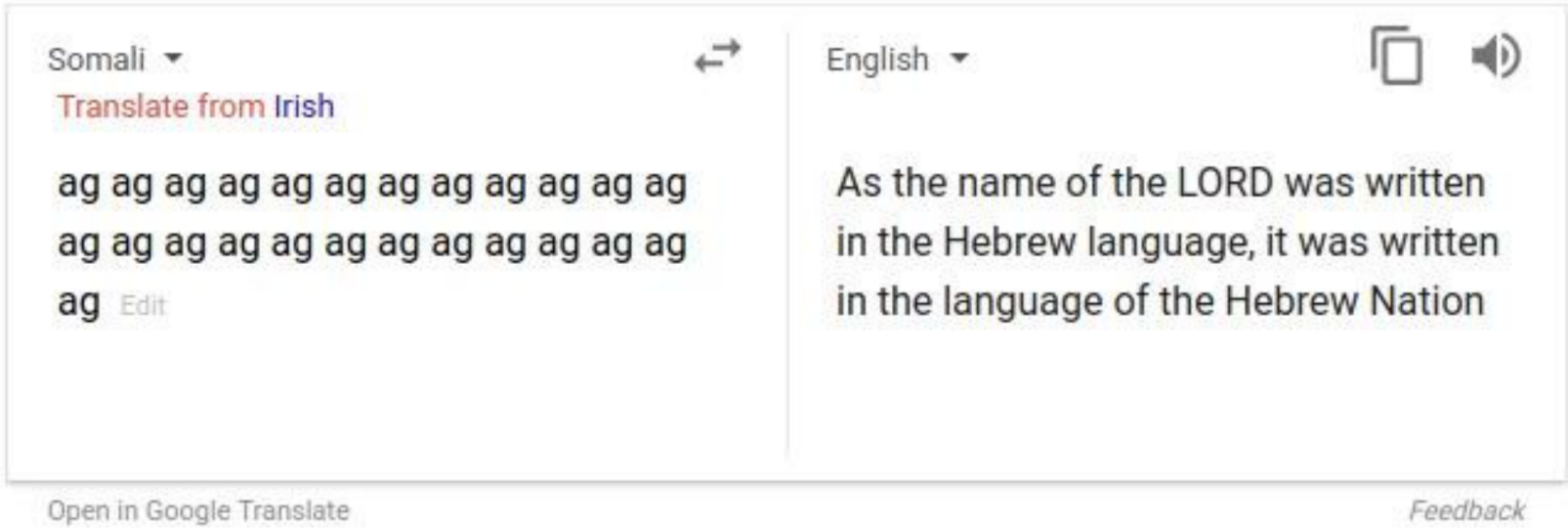
- **Nope!**
- NMT picks up biases in training data



Source: https://hackernoon.com/bias-sexist-or-this-is-the-way-it-should-be-ce1f7c8c683c

So is Machine Translation solved?

- Nope!
- Uninterpretable systems do strange things



Picture source: vice.com/en_uk/article/j5npeg

Explanation: https://www.skynettoday.com/briefs/google-nmt-prophecies

NMT research continues

NMT is the flagship task for NLP Deep Learning

- NMT research has pioneered many of the recent innovations of NLP Deep Learning
- NMT research continues to thrive (WMT shared tasks every year)
- Researchers have found many, many improvements to the "vanilla" seq2seq NMT system we've presented today
- But one improvement is so integral that it is the new vanilla...

ATTENTION

Content

1 Machine Translation (MT)

2 Machine translation evaluation

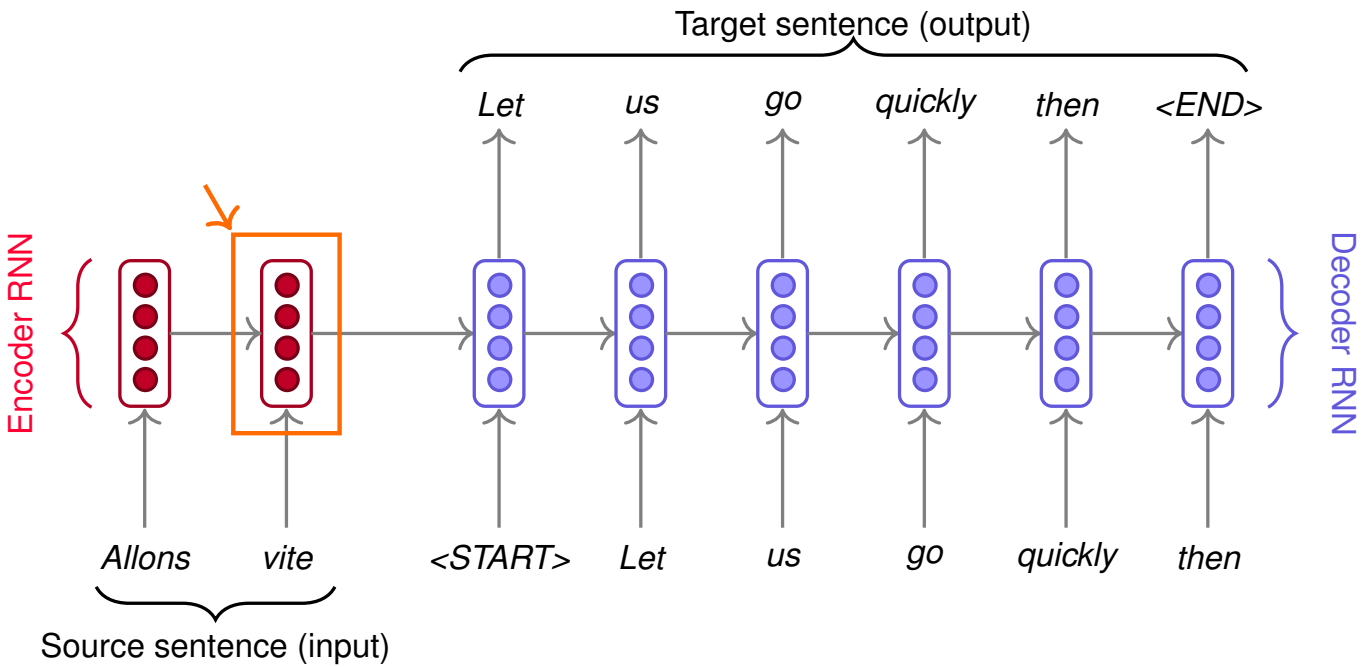
3 Statistical machine translation (SMT)

4 Neural machine translation (NMT) based on RNNs

5 Neural machine translation (NMT) with attention

Sequence-to-sequence: the bottleneck problem

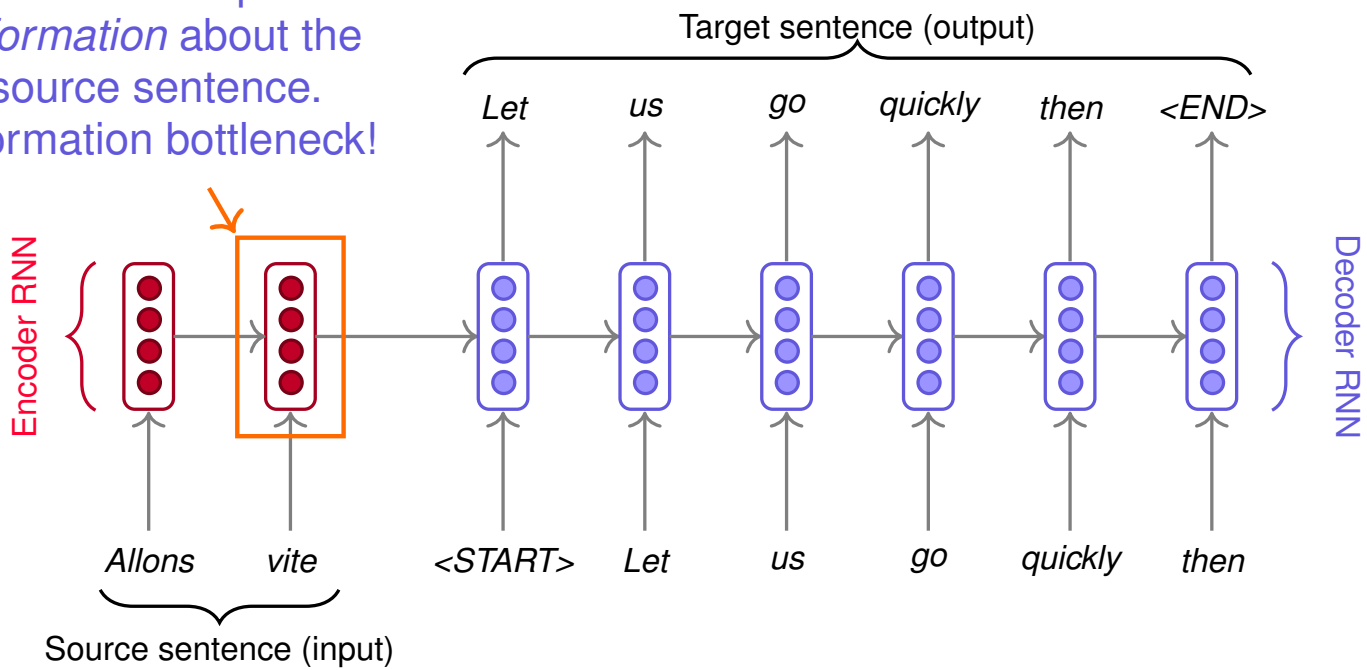
Encoding of the source sentence.



Problems with this architecture?

Sequence-to-sequence: the bottleneck problem

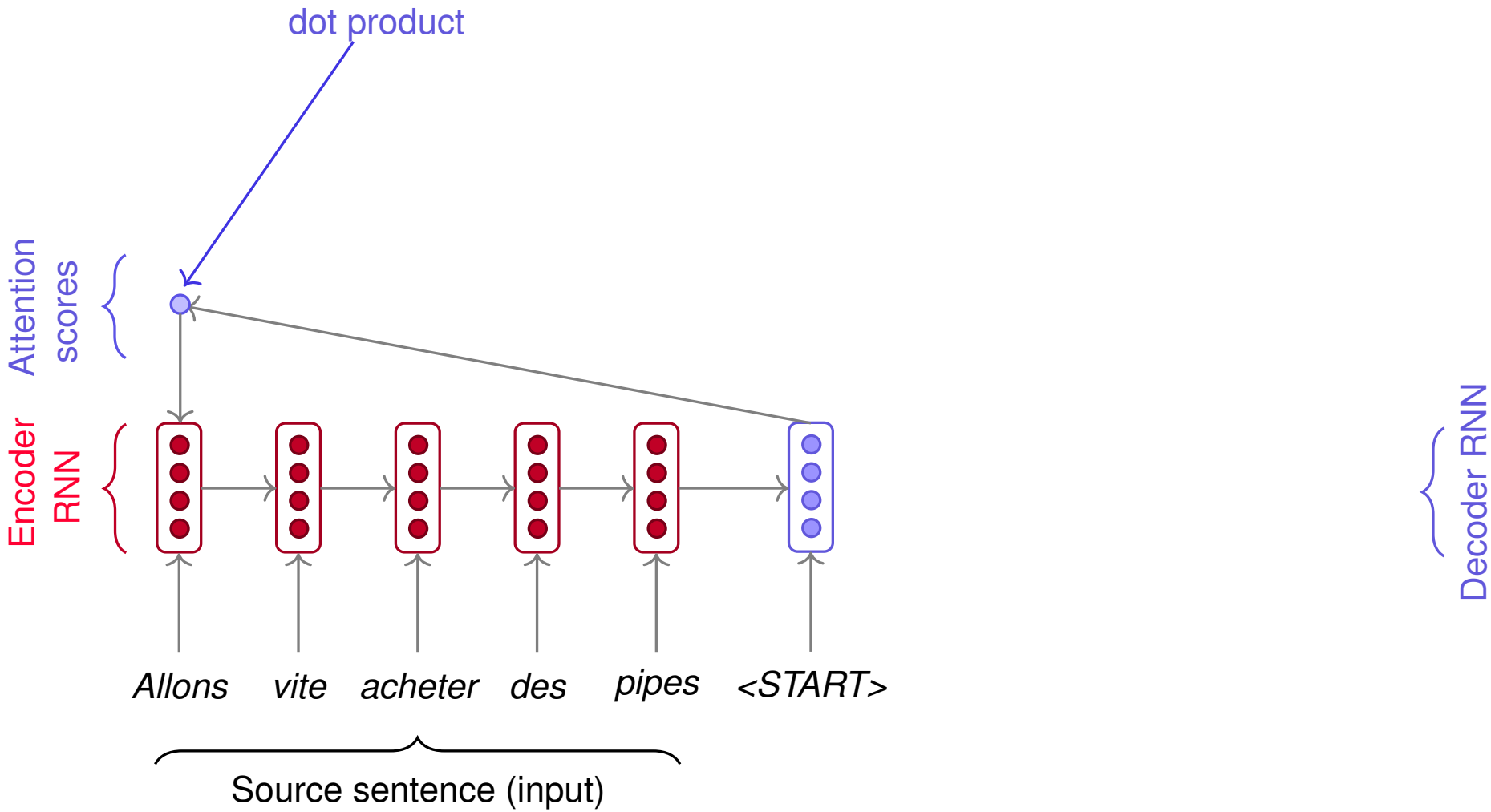
Encoding of the source sentence. This needs to capture *all information* about the source sentence. Information bottleneck!



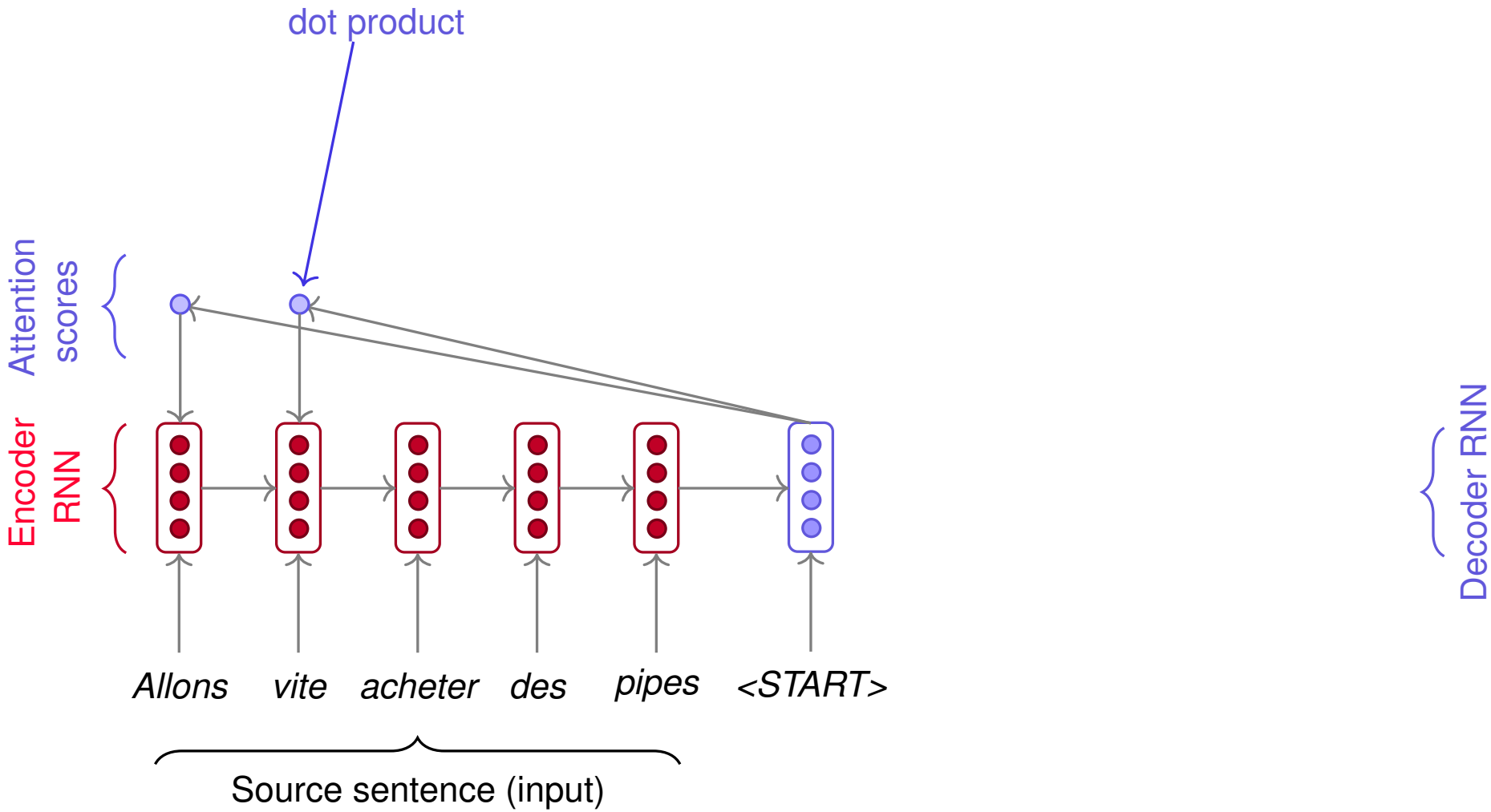
Attention

- Attention provides a solution to the bottleneck problem.
- Core idea: on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence
- First we will show via diagram (no equations), then we will show with equations

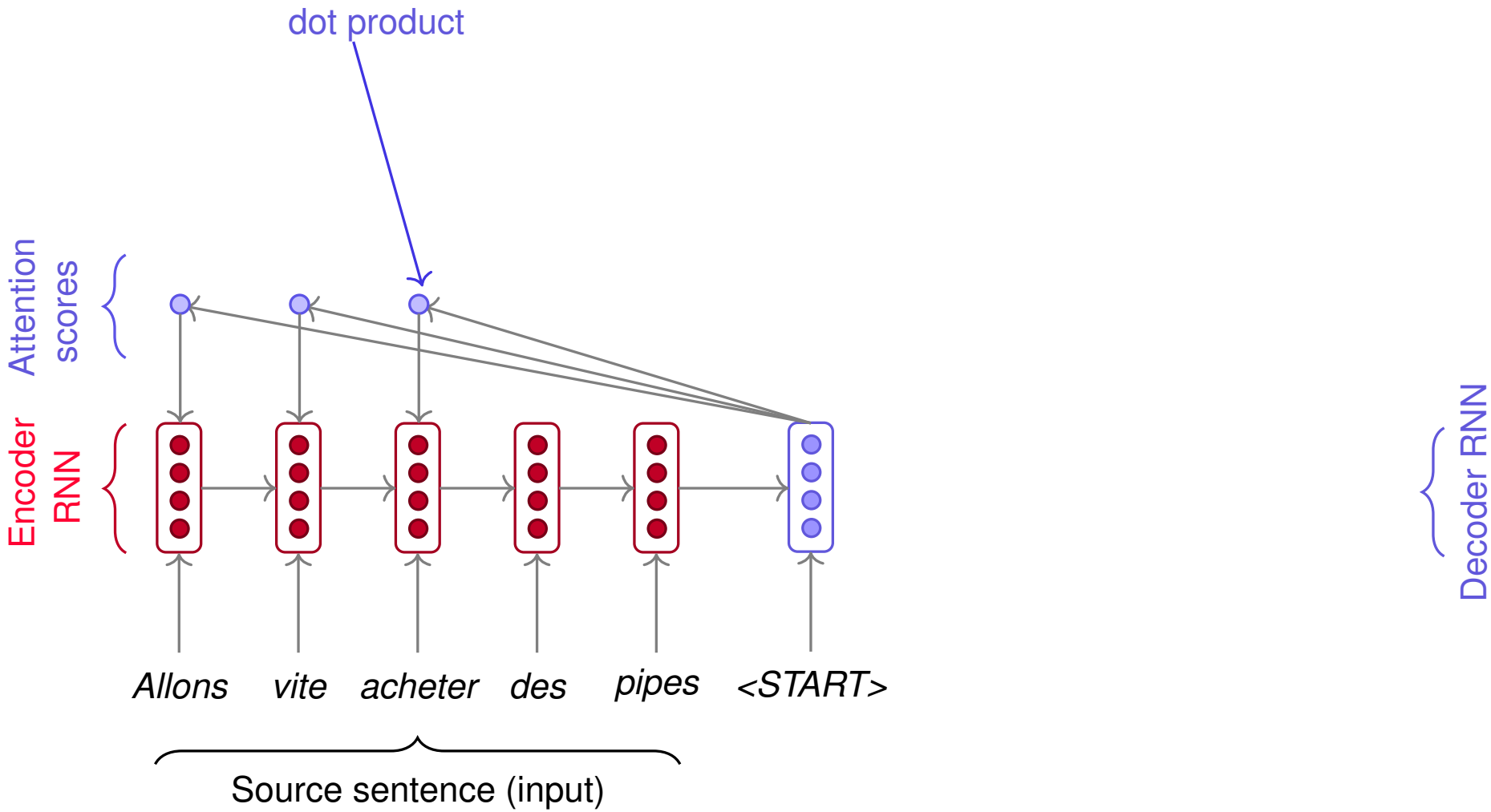
Sequence-to-sequence with attention



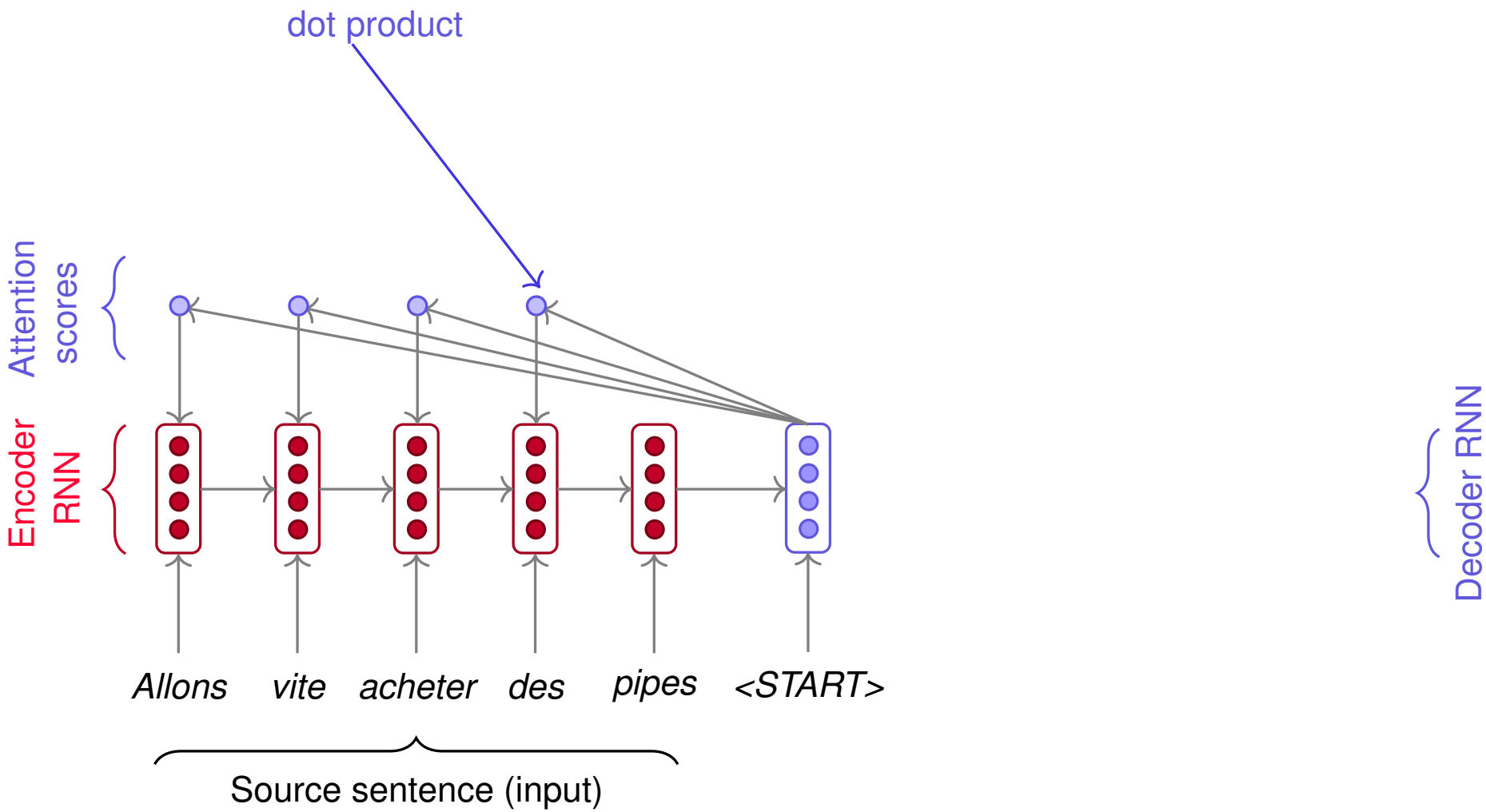
Sequence-to-sequence with attention



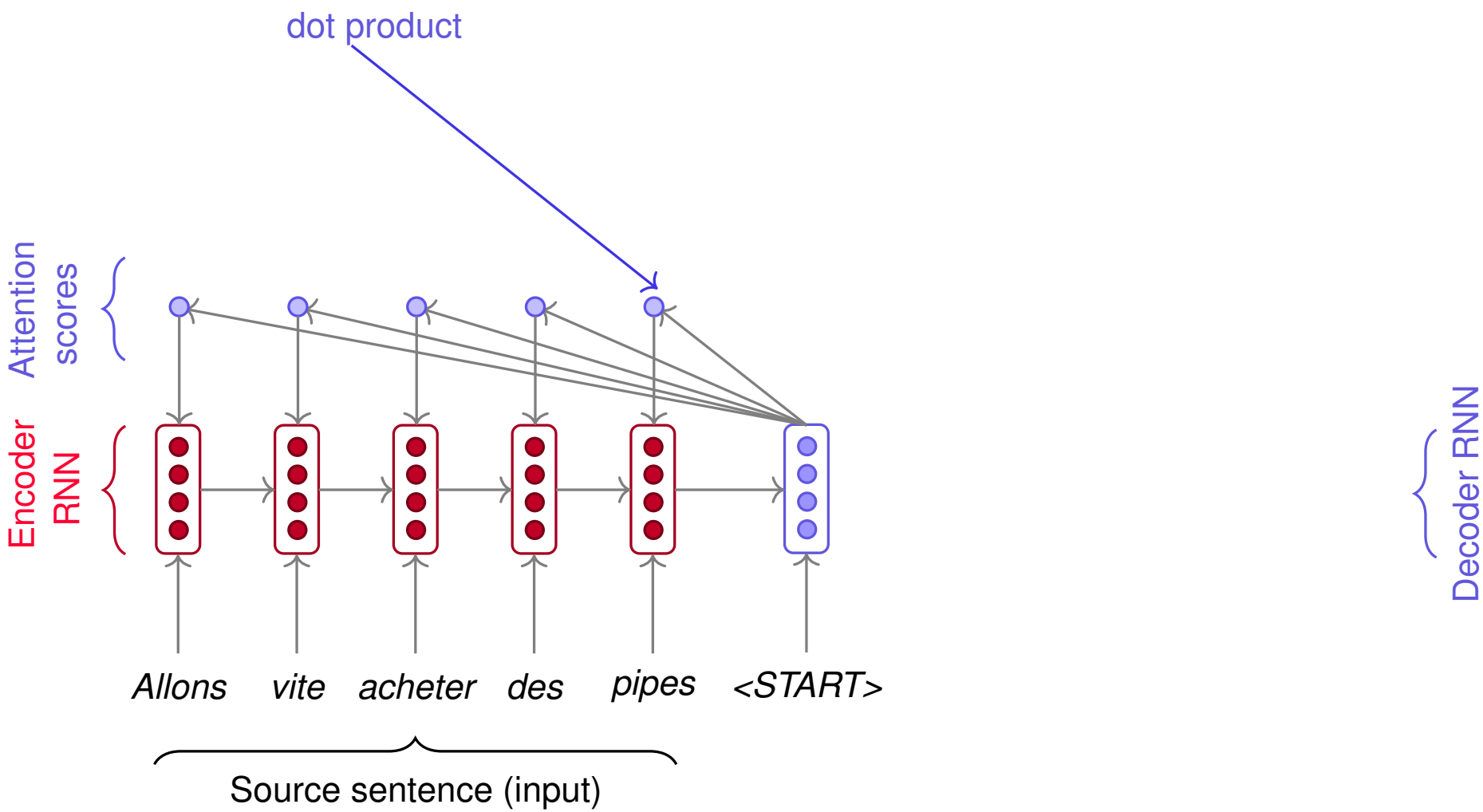
Sequence-to-sequence with attention



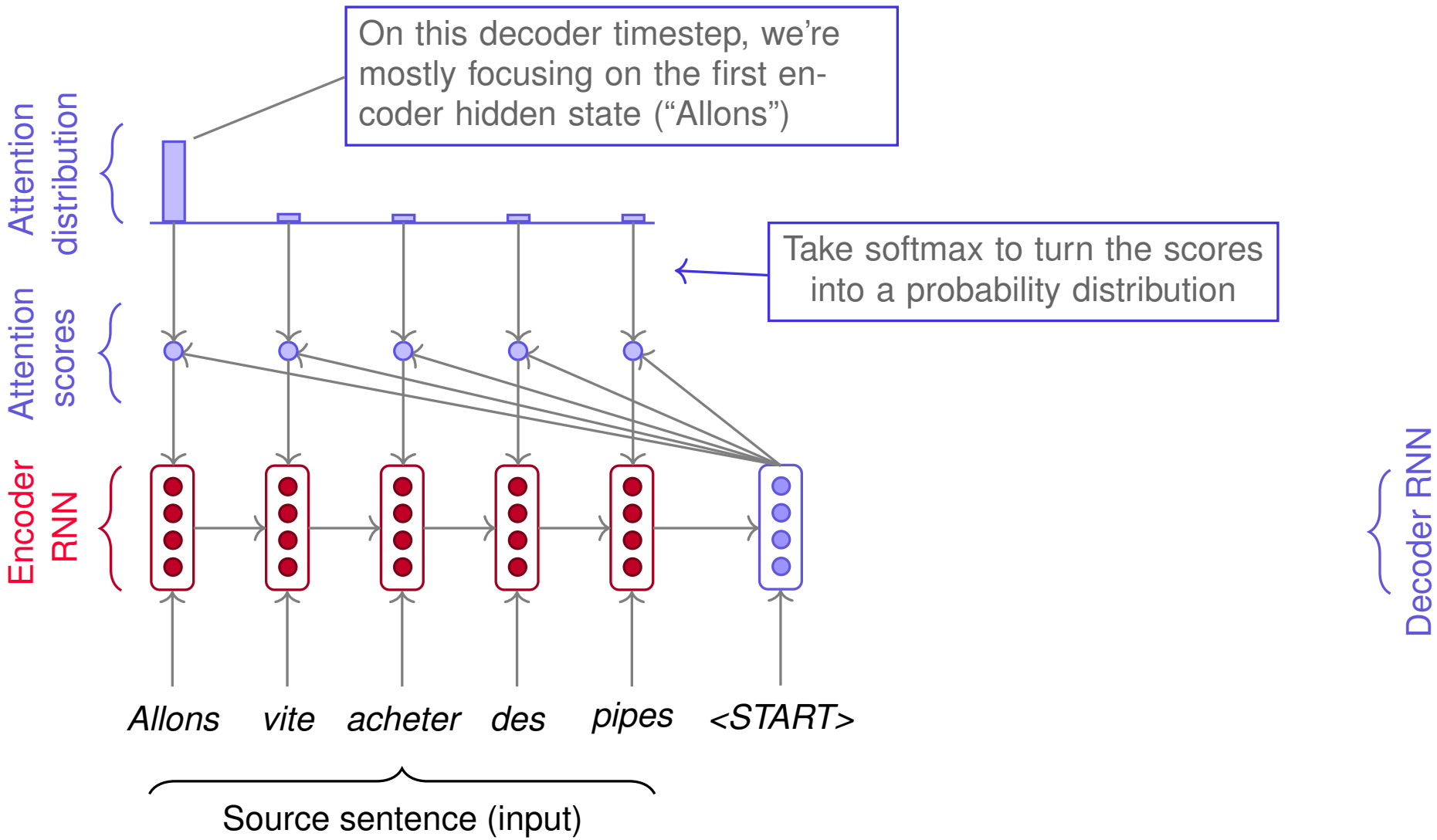
Sequence-to-sequence with attention



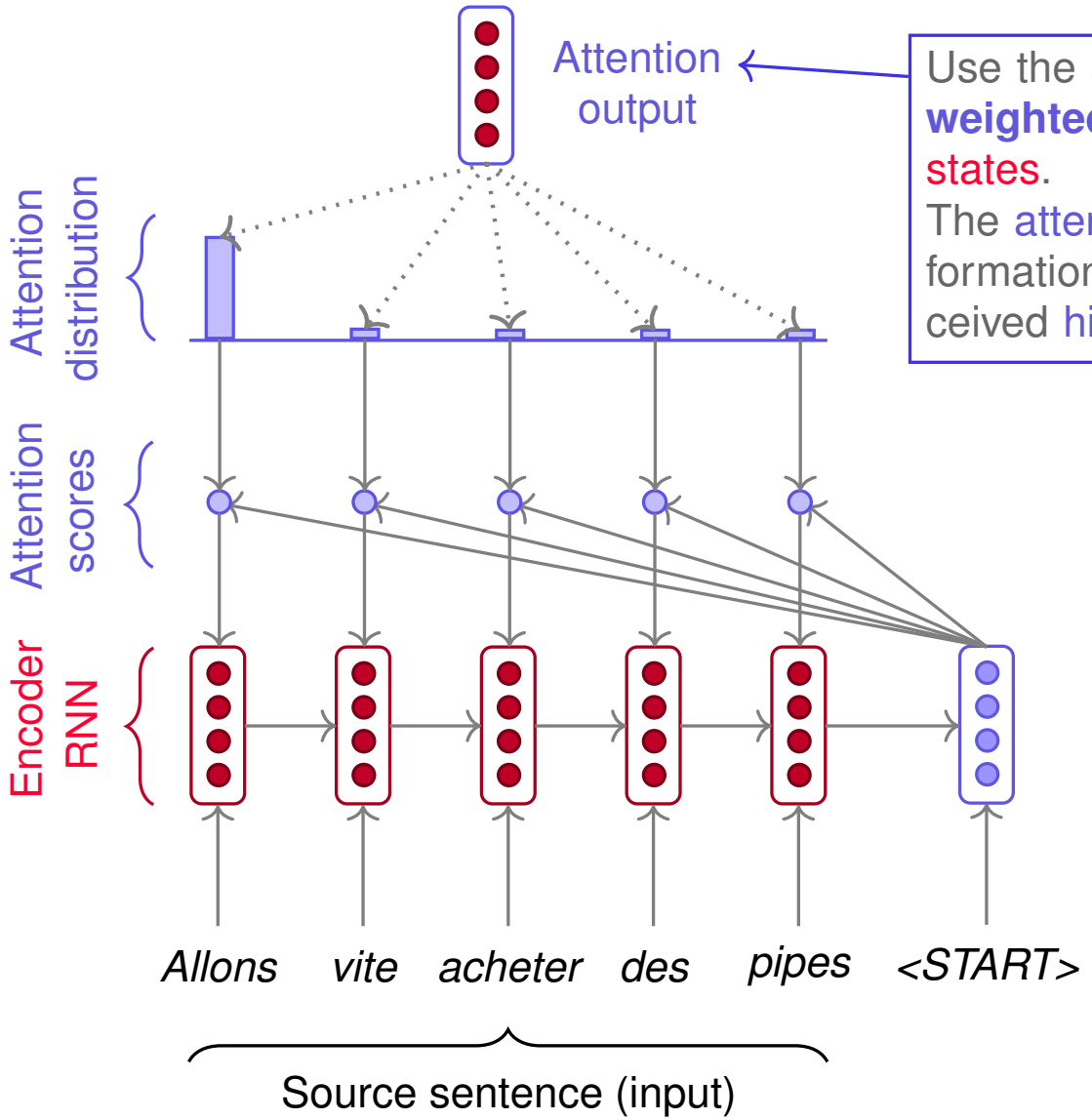
Sequence-to-sequence with attention



Sequence-to-sequence with attention



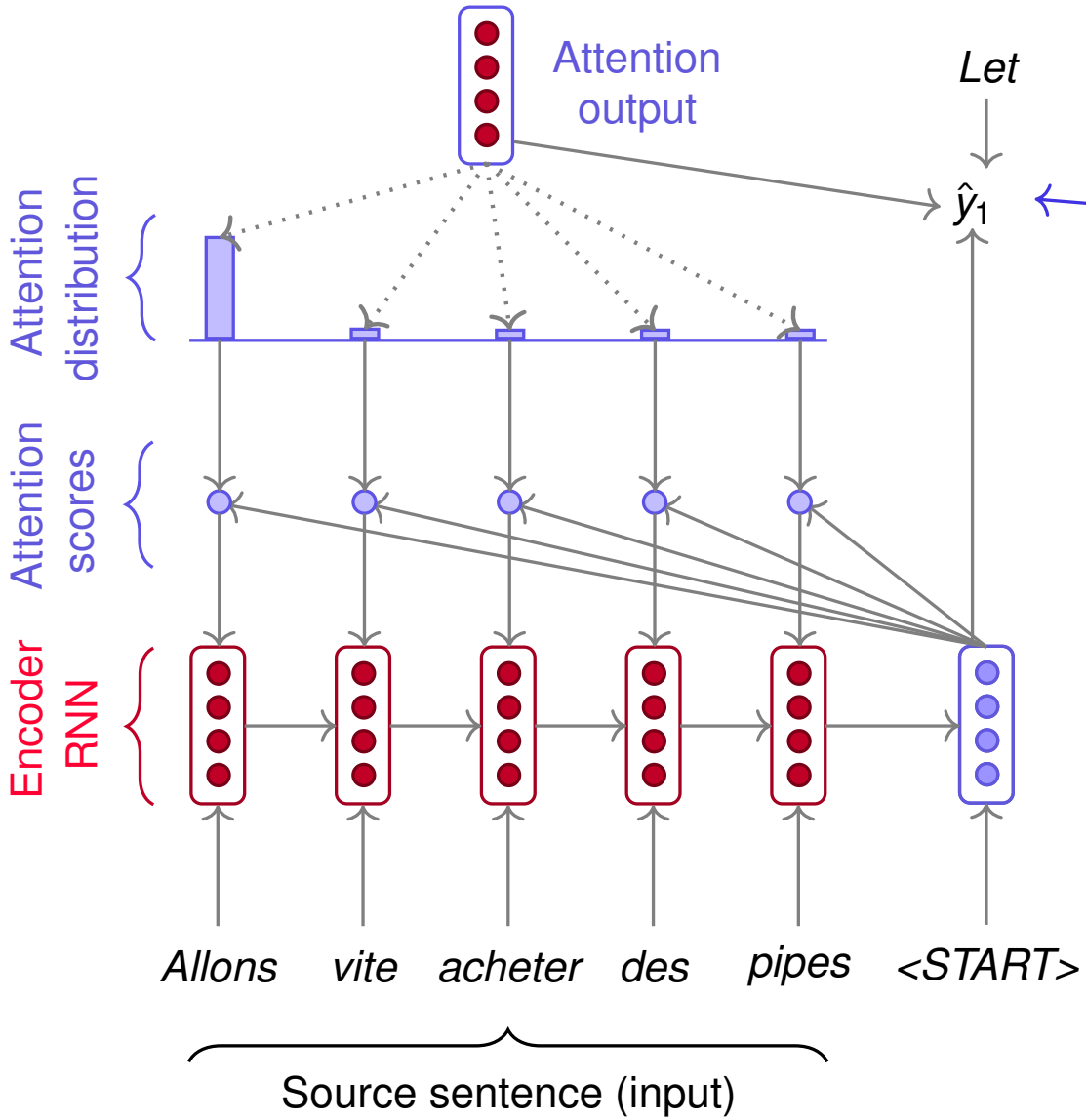
Sequence-to-sequence with attention



Use the attention distribution to take a **weighted sum** of the encoder hidden states.
The attention output mostly contains information from the hidden states that received high attention.

Decoder RNN

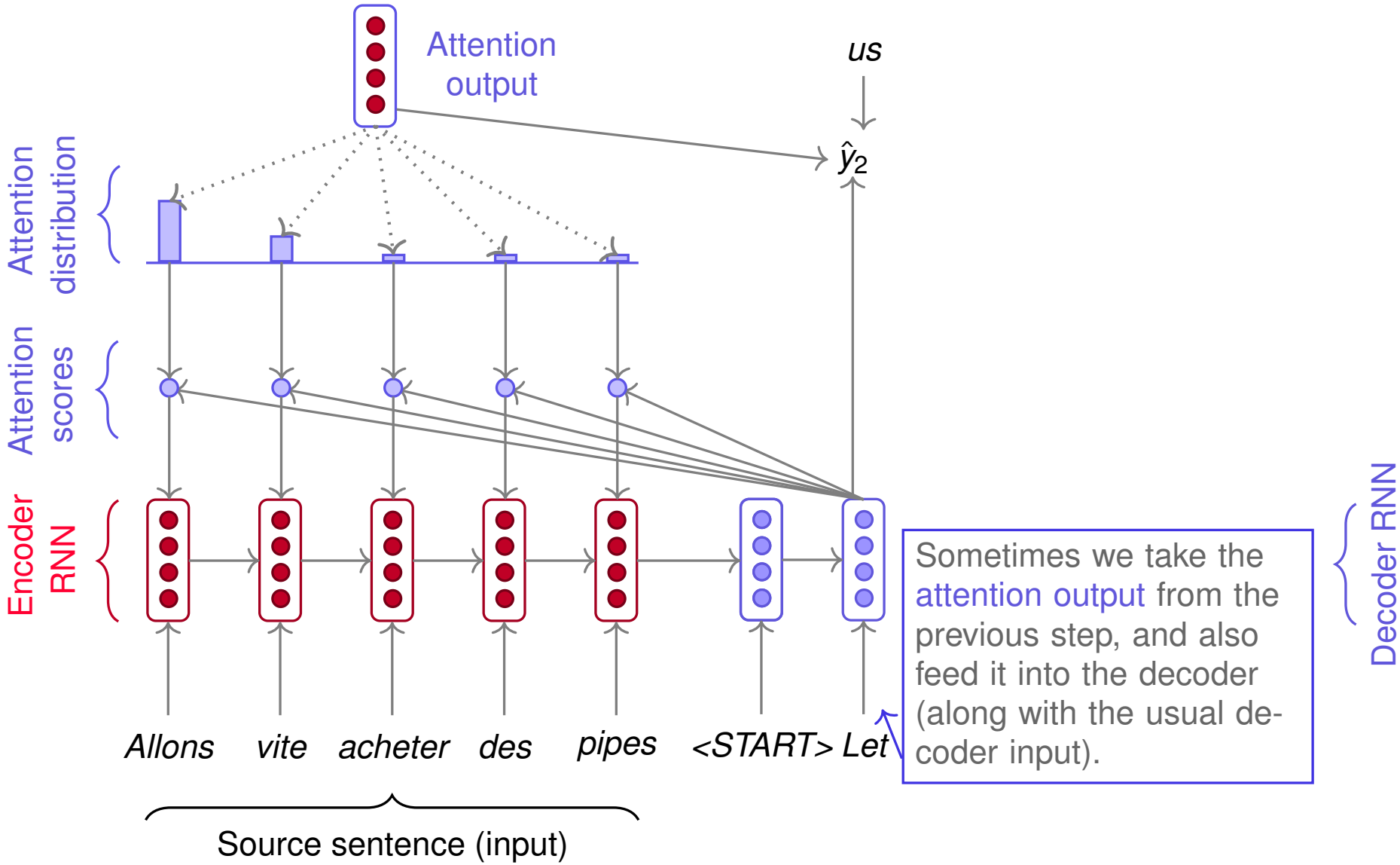
Sequence-to-sequence with attention



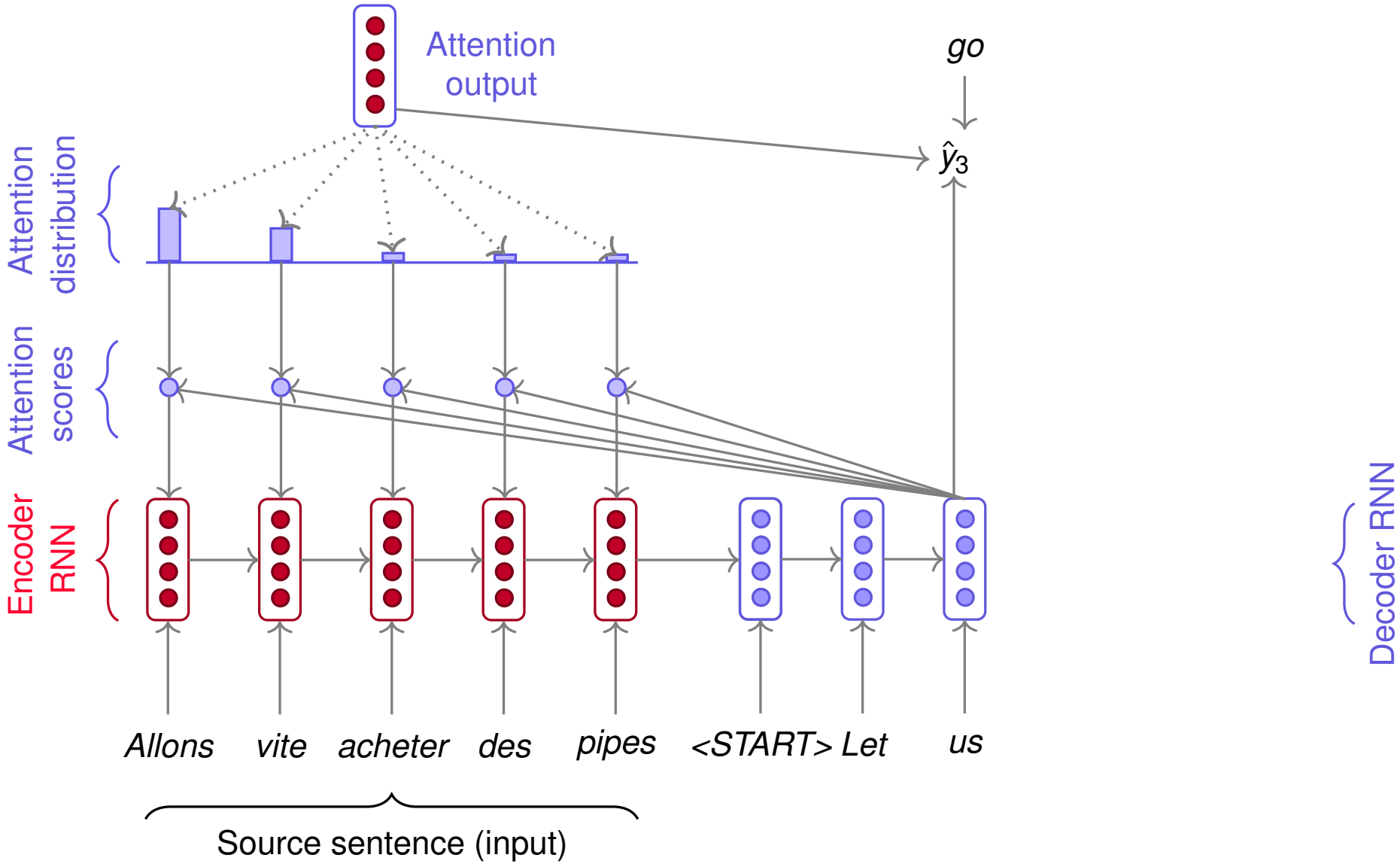
Concatenate attention output with decoder hidden state, then use to compute $\hat{y}_1$ as before

Decoder RNN

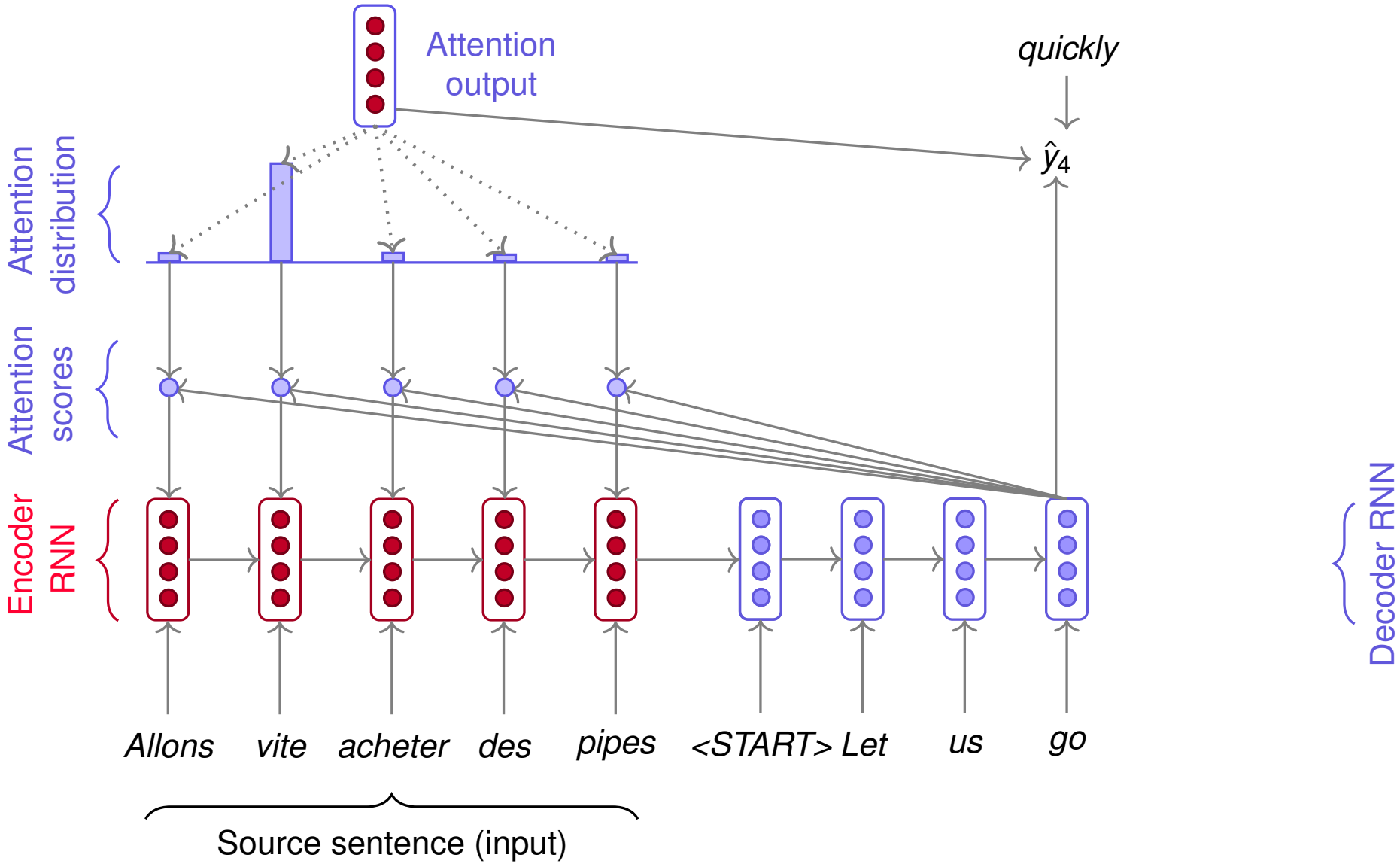
Sequence-to-sequence with attention



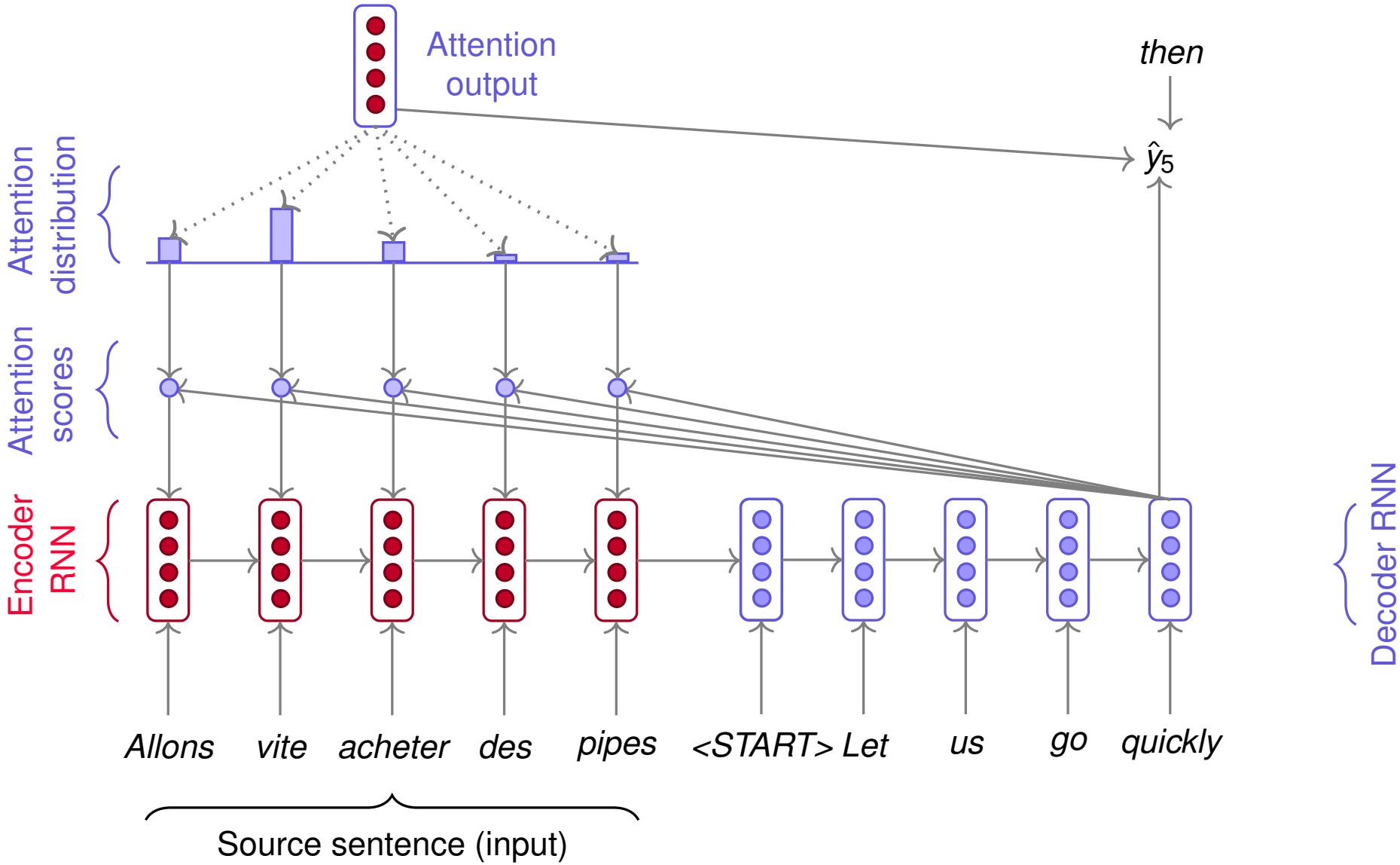
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention

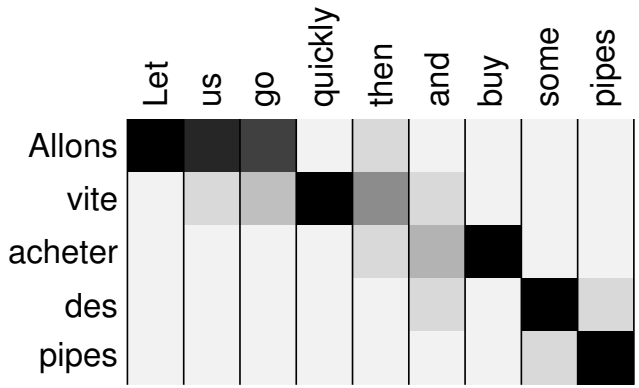


Attention: in equations

- We have encoder hidden states $h_1, \ldots, h_N \in \mathbb{R}^h$
- On timestep t, we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step: $e^t = [\, s_t^T h_1, \ \ldots, \ s_t^T h_N \,] \in \mathbb{R}^N$
- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1):
  $\alpha^t = \mathrm{softmax}(e^t) \in \mathbb{R}^N$
- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t:
  $a_t = \sum_{i=1}^{N} \alpha_i^t h_i \in \mathbb{R}^h$
- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model:
  $[\, a_t; \ s_t \,] \in \mathbb{R}^{2h}$

Attention is great

- Attention significantly improves NMT performance
  - It's very useful to allow decoder to focus on certain parts of the source
- Attention solves the bottleneck problem
  - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
  - Provides shortcut to faraway states
- Attention provides some interpretability
  - By inspecting attention distribution, we can see what the decoder was focusing on
  - We get (soft) alignment for free!
  - This is cool because we never explicitly trained an alignment system
  - The network just learned alignment by itself



Attention is a general Deep Learning technique

- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- However: You can use attention in many architectures (not just seq2seq) and many tasks (not just MT)
- More general definition of attention:
  - Given a set of vector values, and a vector query, attention is a technique to compute a weighted sum of the values, dependent on the query.
  - We sometimes say that the query *attends to* the values.
  - For example, in the seq2seq + attention model, each decoder hidden state (query) attends to all the encoder hidden states (values).

Attention is a general Deep Learning technique

More general definition of attention:

Given a set of vector values, and a vector query, attention is a technique to compute a weighted sum of the values, dependent on the query.

Intuition:

- The weighted sum is a selective summary of the information contained in the values, where the query determines which values to focus on.
- Attention is a way to obtain a fixed-size representation of an arbitrary set of representations (the values), dependent on some other representation (the query).

Content

1. Machine Translation (MT)
2. Machine translation evaluation
3. Statistical machine translation (SMT)
4. Neural machine translation (NMT) based on RNNs
5. Neural machine translation (NMT) with attention